
1. Локальный вариант распределенной системы

Введение

Работа 1. «Толстый» клиент с БД

Цель работы — построение распределенной системы (локальный вариант).

Распределенные (сетевые) системы в последнее время получают все более широкое распространение. Прежде всего с технологией клиент-сервер.

В качестве СУБД (рис. 1.1) используется InterBase, работающая в среде Builder C++.



Рис. 1.1

Вариант реализации технологии клиент-сервер

Сочетание (рис. 1.1) более удобно для систем средней размерности. Однако оно обладает хорошими методическими свойствами и потому далее рассматривается более подробно.

СУБД и среда, если клиентов несколько, могут располагаться на разных компьютерах (рис. 1.2).

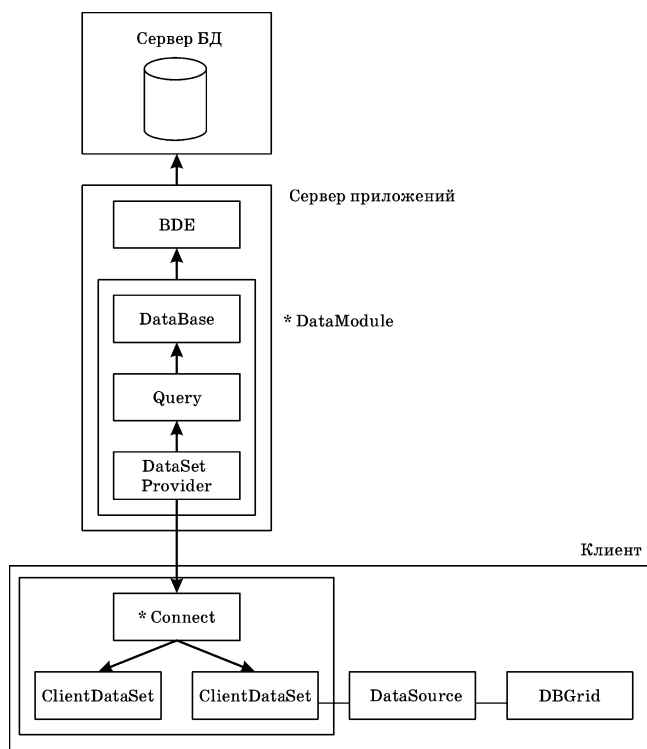


Рис. 1.2

Детализированная многоуровневая структура режима клиент-сервер

В данном случае рассматривается один клиент, поэтому клиент и сервер расположены на одном компьютере (рис. 1.1).

Описание работы

Задание. Имеется склад материалов (М, М2, М3), снабжающий производство, выпускающее изделия И1, И2. Каждое из изделий состоит из деталей Д1, Д2 и Д3, количество которых в изделиях (входимость) известно. Известны и нормы расхода материалов на каждую деталь. Задан — на протяжении месяца — ежедневный план выпуска изделий, который склад должен обеспечить материалами. Длительность технологического цикла (производства) изделий t_d — 3 дня. Рекомендуется создать страховой запас материалов на складе на один день. Склад периодически заказывает материалы поставщикам. Время τ выполнения заказа r по первому и третьему материалам — 2 дня, по второму — 1 день. Поставки материалов производятся по первому и третьему материалам — каждый третий день, по второму — каждый четвертый день.

Необходимо ежедневно определить (решить задачи, выделенные из ранее перечисленных):

31. запуск материалов под план производства;
32. поставки материалов;
33. заказы на материалы;

34. движение материалов на складе по дням.

Исходные данные. Пусть данные упорядочены в виде системы таблиц: материалы, детали, изделия, нормы (расхода), входимость (применяемость), план (выпуска изделий).

Материалы

Шифр_материала	Название материала	Единица измерения	Цена	Интервал поставок	Время выполнения
M1	сталь	кг	23	3	2
M2	чугун	кг	12	4	1
M3	железо	кг	15	3	2

Детали

Шифр_детали	Название детали	Единица измерения
D1	втулка	шт.
D2	фланец	шт.
D3	палец	шт.

Нормы

Шифр_материала	Шифр_детали	Единица измерения	Норма расхода
M1	D1	кг/шт.	2,1
M1	D2	кг/шт.	1,8
M2	D1	кг/шт.	3,4
M2	D3	кг/шт.	4,3
M3	D2	кг/шт.	1,5
M3	D3	кг/шт.	3,7

Входимость (применяемость)

Шифр_детали	Шифр_изделия	Ед_изм	Количество
D1	I1	шт./изд.	7
D2	I1	шт./изд.	9
D2	I2	шт./изд.	8
D3	I2	шт./изд.	5

План выпуска изделий

Дата	Шифр_изделия	Количество изделий
1	I1	7
2	I1	11
3	I1	8
4	I1	9
5	I1	12
6	I1	13
7	I1	9
8	I1	8
9	I1	7
10	I1	9
11	I1	14
12	I1	11
13	I1	8
14	I1	4

15	И1	7
16	И1	5
17	И1	8
18	И1	9
19	И1	10
20	И1	14
21	И1	17
22	И1	12
23	И1	11
24	И1	10
25	И1	8
1	И2	5
2	И2	12
3	И2	8
4	И2	7
5	И2	4
6	И2	3
7	И2	15
8	И2	4
9	И2	8
10	И2	7
11	И2	4
12	И2	20
13	И2	25
14	И2	14
15	И2	11
16	И2	10
17	И2	15
18	И2	5
19	И2	7
20	И2	10
21	И2	8
22	И2	4
23	И2	12
24	И2	15
25	И2	18

Для работы с БД следует сформировать интерфейс в виде меню, чтобы с его помощью получить доступ как к таблицам, так и к запросам.

Порядок выполнения работы

В соответствии с заданием база данных имеет схему связей, показанную на рисунке 1.3.

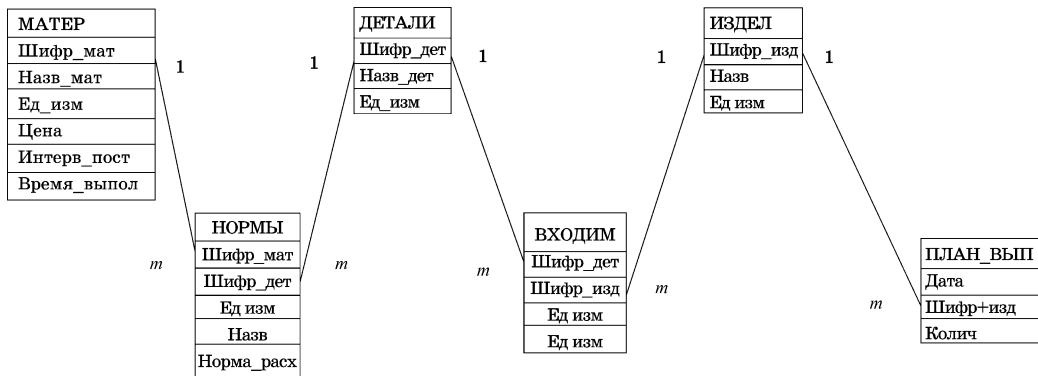


Рис. 1.3

Схема базы данных

Построение системы проводится в три этапа.

1. Создание системы таблиц с их связями.
2. Формирование интерфейса пользователя.
3. Реализация алгоритма приложений.

Создание системы таблиц с их связями

Схема реализации системы таблиц представлена на рисунке 1.4. Из нее видно, что формируются сами таблицы; триггеры (попарно на изменение и удаление), обеспечивающие связь таблиц.

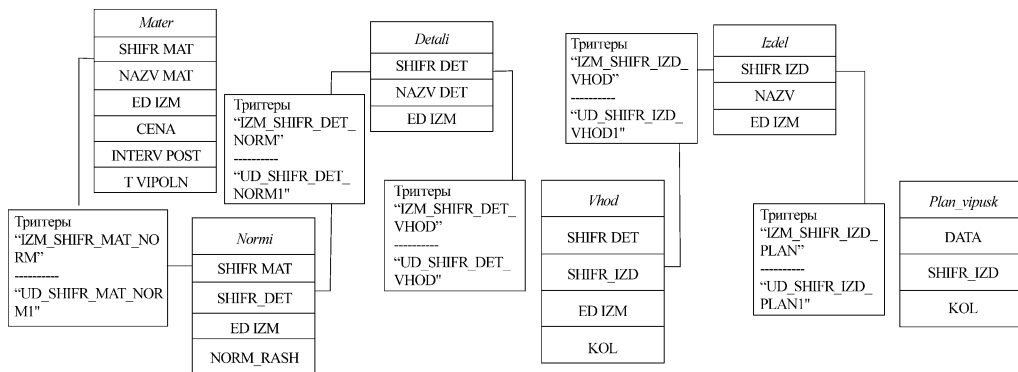


Рис. 1.4

Связи в БД

Таблицы формируются в СУБД InterBase.

Пример таблицы.

//Таблица Материалы

CREATE TABLE "MATER"

```
(
  "SHIFR_MAT"    CHAR(2) CHARACTER SET WIN1251 NOT NULL,
  "NAZV_MAT"     CHAR(20) CHARACTER SET WIN1251,
  "ED_IZM"       CHAR(2) CHARACTER SET WIN1251,
  "CENA"         INTEGER,
  "INTERV_POST"  INTEGER,
```

```
"T_VIPOLN"      INTEGER,
"RASN"          INTEGER,
PRIMARY KEY ("SHIFR_MAT")
);
```

Пример триггера.

//триггер на изменение

```
CREATE TRIGGER "IZM_SHIFR_MAT_NORM" FOR "MATER"
ACTIVE BEFORE UPDATE POSITION 0
as
begin
if(old.SHIFR_MAT<>new.SHIFR_MAT) then
update NORMI
set SHIFR_MAT=new.SHIFR_MAT
where SHIFR_MAT=old.SHIFR_MAT;
end
^
```

```
SET TERM ^ ;
```

```
CREATE TRIGGER "UD_SHIFR_MAT_NORM1" FOR "MATER"
ACTIVE AFTER DELETE POSITION 0
as
begin
delete from NORMI
where NORMI.SHIFR_MAT=MATER.SHIFR_MAT;
end
^
```

```
COMMIT WORK ^
```

```
SET TERM ;^
```

Формирование интерфейса пользователя

Предложено выполнить интерфейс пользователя в таком виде, как на рисунке 1.5.

Главная	Таблицы	Запросы
Начать	Материалы	Потребность
Завершить	Детали	Поставки
	Изделия	Заказ
	План	Движение
	Нормы	
	Входимость	

Рис. 1.5

Интерфейс пользователя

Реализация алгоритма приложений

Алгоритм приложения для каждого элемента меню состоит из двух частей:

- запрос пользователя, выполняемый в среде Builder C++, и обращение к соответствующей хранимой процедуре СУБД InterBase;
- выполнение хранимой процедуры и передача запрошенных данных в среду с отображением результата на экране среды.

Покажем это на примере.

Клиент (Builder C++) после нажатия элемента меню Запросы/Поставки вызывает процедуру в составе Delphi-клиента (рис. 1.6).

```
procedure TForm1.N11Click(Sender: TObject);  
begin  
  DBGrid1.Visible:=True;  
  Label1.Caption:='ПОСТАВКИ';  
  IBQuery1.Close;  
  IBQuery1.SQL.Clear;  
  IBQuery1.SQL.Add('select * from POST');  
  IBQuery1.Open;  
end;
```

Рис. 1.6

Процедура на клиенте

Процедура, в свою очередь, вызывает хранимую процедуру POST в сервере (рис. 1.7).

```
COMMIT WORK;
SET AUTODDL OFF;
SET TERM ^;

/* Stored procedures */

CREATE PROCEDURE "POST"
RETURNS
(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_POSTAV" DOUBLE PRECISION
)
AS
BEGIN EXIT; END ^

ALTER PROCEDURE "POST"
RETURNS
(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_POSTAV" DOUBLE PRECISION
)
AS
begin
for select POTREBN3.OUT_DATA, POTREBN3.OUT_SHIFR_MAT,
IIF(POTREBN3.OUT_MODUL, 0, AVERADD.OUT_AVERAGE)
from POTREBN3, AVERADD
where POTREBN3.OUT_SHIFR_MAT=AVERADD.OUT_SHIFR_MAT
ORDER BY POTREBN3.OUT_DATA, POTREBN3.OUT_SHIFR_MAT
INTO :OUT_DATA, :OUT_SHIFR_MAT, :OUT_POSTAV
DO
SUSPEND;
end
^

SET TERM ; ^
COMMIT WORK;
SET AUTODDL ON;
```

Рис. 1.7

Хранимая процедура в сервере

Хранимая процедура обращается к базе данных, формирует ответ и отправляет его клиенту.

Результат получает вид, показанный на рисунке 1.8.

OUT_DATA	OUT_SHIFFR_MAT	OUT_NAZV_MAT	OUT_POTREB
1	M1	СТАЛЬ	288,3
1	M2	ЧУГУН	274,1
1	M3	ЖЕЛЕЗО	247
2	M1	СТАЛЬ	512,7
2	M2	ЧУГУН	519,8

Рис. 1.8

Результат запроса

Работа с системой

1. Запустить Builder C++, нажав «зеленый треугольник» в верхнем интерфейсе среды или выбрав меню среды Run/Run. Появляется сформированное ранее меню (рис. 1.9).

2. Иллюстрируем работу системы при нажатии клиентом элемента меню Потребность.

3. Клиентом через меню (рис. 1.9) и программы Builder C++ (рис. 1.6) осуществляется запрос к серверу (хранимые процедуры (рис. 1.7) и база данных). Ответ сервера фиксируется на экране компьютера клиента.

4. Последовательно просмотреть через меню (рис. 1.9) таблицы и запросы. Возможно, потребуется указать пароль masterkey.

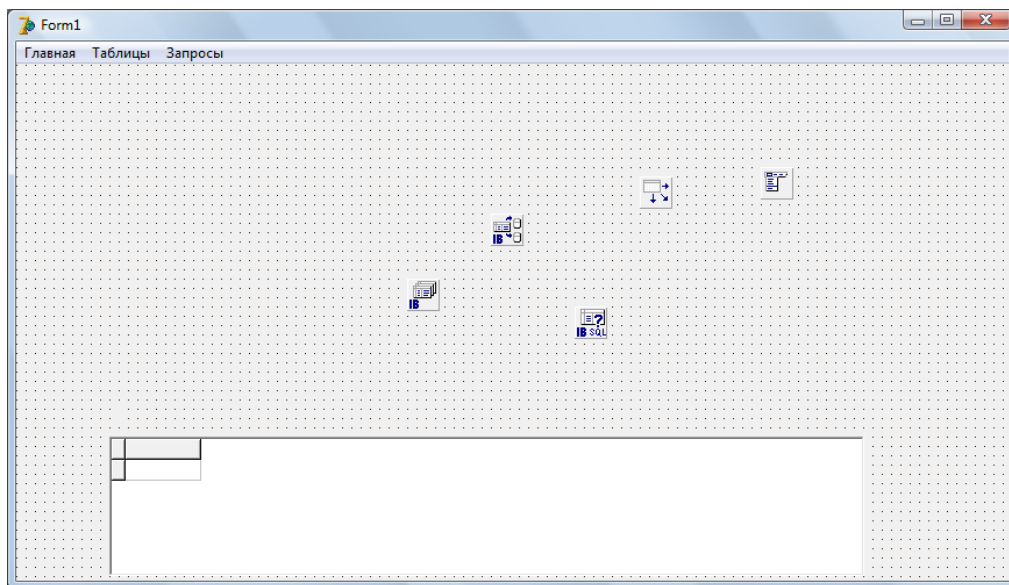


Рис. 1.9

Меню системы

Содержание отчета

1. Титульный лист.
2. Задание.
3. Введение.
4. Математическое описание задачи.
5. Схема связей базы данных.
6. Примеры команд таблиц, триггеров, SQL-запросов.
7. Экранные формы интерфейса пользователя, таблицы и запроса.
8. Заключение.

2. Генератор числовых данных

Работа 2. Генератор числовых данных

Введение

Цель работы — формирование числовых данных для проверки работоспособности распределенной системы.

Для работы с системой необходимы числовые данные, представляющие реальные данные. Их можно получить из реальной системы, что проблематично прежде всего из-за человеческого фактора. Для проверки работоспособности системы данные следует сгенерировать. В предыдущей работе данные были произвольны. Поскольку в системе предполагается оптимальный режим, данные следует строить по определенным правилам. Данные нужны для трехуровневой системы (рис. 2.1). В данной работе рассмотрен только диспетчерский уровень.

Уровни

$h = 3$

Руководитель

$h = 2$

Диспетчер

$h = 1$

Цехи

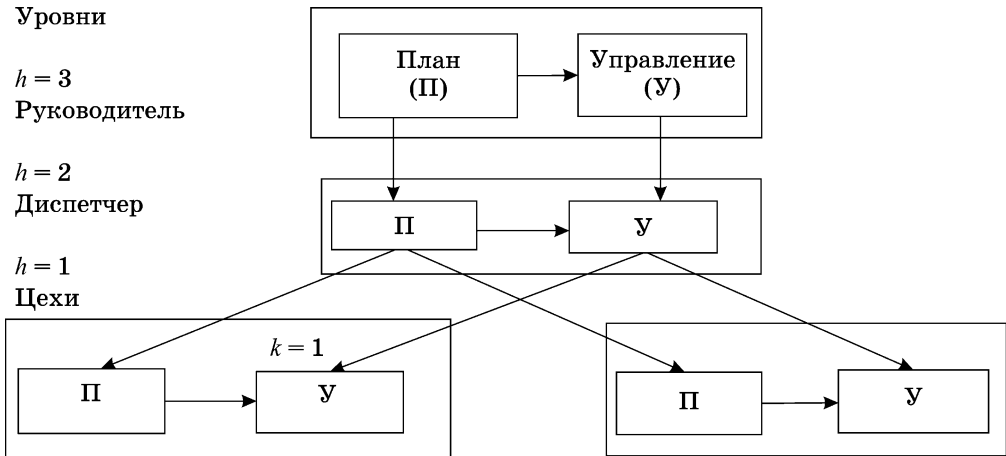


Рис. 2.1

Структура трехуровневой системы

Описание работы

Проведем реализацию положений. Генерация проводится для уровня диспетчера.

Описание диспетчера имеет вид:

$$\sum_{i=0}^{M_1} \mathbf{D}_{11}^m \mathbf{p}_i(t) \leq \mathbf{b}^m(0),$$

$$\sum_{i=0}^{N-1} \mathbf{p}_k(t_i) \leq \mathbf{P}(T),$$

$$\mathbf{D}_k^\Psi \mathbf{p}_k(t_{i+1}) \leq \mathbf{b}_k^\Psi(t_i),$$

$$i = 0, N = 1, t_i = iv, t_0 = 0, T = Nv,$$

$$\mathbf{D}_k^m \mathbf{p}_k(t_{i+1}) \leq \mathbf{p}_{k-1}(t_i),$$

$$G = \sum_{k=1}^K G_k \rightarrow \max,$$

где $\mathbf{p}$ — вектор-столбцы (планового) незавершенного производства и ежедневного плана; $\mathbf{R}$ — вектор-столбец спроса; $\mathbf{D}$ — матрица норм расходов ресурсов; $\mathbf{b}$ — вектор-столбец наличного количества ресурсов; $\mathbf{b}^m(0)$ — вектор количества материальных ресурсов, которыми располагает уровень $h = 3$; $\Delta \mathbf{b}$ — поступление ресурсов; $\mathbf{P}$ — вектор-столбец плана уровня $h = 3$; $\mathbf{F}$ — вектор-строка прибыли от выпуска единицы продукции; $\mathbf{A}, \mathbf{B}, \mathbf{C}$ — единичные матрицы соответствующих размерностей; v, T — минимальный интервал времени и время моделирования; $m = 1, M$ — виды материальных ресурсов; $\psi = 1, \Psi$ — виды прочих ресурсов; $i = 1, I$ — моменты времени; $k = 1, K$ — номер подразделения.

В генераторе используются другие обозначения. Математическое описание отдельного элемента уровня $h = 2$ с использованием задачи линейного программирования имеет вид:

$$\mathbf{PN} \leq \mathbf{P} \leq \mathbf{PV},$$

$$\mathbf{BN} \leq \mathbf{D}\mathbf{P} \leq \mathbf{BV},$$

$$G = \mathbf{F}\mathbf{P} \rightarrow \max,$$

где $\mathbf{P}$ — вектор искомого плана; $\mathbf{PN}, \mathbf{PV}$ — векторы нижнего и верхнего ограничений на план; $\mathbf{D}$ — матрица норм расходов ресурсов; $\mathbf{F}$ — вектор прибыли от единицы плановой продукции; $\mathbf{BN}, \mathbf{BV}$ — векторы нижней и верхней границ ресурсов. Выходом является вектор $\mathbf{P}$, входом — вектор $\mathbf{B} = (\mathbf{BN}, \mathbf{BV})$. Это прямая задача линейного программирования: по $\mathbf{PV}, \mathbf{D}, \mathbf{BV}, \mathbf{F}$ найти $\mathbf{P}$.

При моделировании технологической линии необходимо решать обратную задачу: по $\mathbf{P}, \mathbf{D}, \mathbf{DV}, \mathbf{F}$ найти $\mathbf{BV} = \mathbf{B}$ с учетом условий:

$$(0, \mathbf{BV})_k = (0, \mathbf{PV})_{k-1}, \mathbf{BV}_k = \mathbf{PV}_{k-1}, \mathbf{P}_k = \mathbf{B}_{k-1} = \mathbf{D}_{k-1} \cdot \mathbf{P}_{k-1},$$

где k — номер структурного элемента (последний элемент $k = 1$, первый — $k = K$).

За основу взята обратная задача Р. Габасова для отдельного структурного элемента (цеха). Генерация данных для диспетчера осуществляется соединением элементов.

Порядок выполнения работы

Создание генератора

1. Работу выполнить на языке программирования Java.

2. В качестве основы для прикладной модели цепочки принята модель отдельного элемента. Математическое его описание для уровня $h = 2$ с использованием задачи линейного программирования имеет вид:

$$\begin{aligned}PN &\leq P \leq PV, \\ BN &\leq DP \leq BV, \\ G &= FP \rightarrow \max,\end{aligned}$$

где P — вектор искомого плана; PN , PV — векторы нижнего и верхнего ограничений на план; D — матрица норм расходов ресурсов; F — вектор прибыли от единицы плановой продукции; BN , BV — векторы нижней и верхней границ ресурсов. Выходом является вектор P , входом — вектор $B = (BN, BV)$. Это прямая задача линейного программирования: по PV , D , BV , F найти P .

При моделировании технологической линии необходимо решать обратную задачу: по P , D , DV , F найти $BV=B$ с учетом условий:

$$(0, BV)_k = (0, PV)_{k-1}, BV_k = PV_{k-1}, P_k = B_{k-1} \cdot P_{k-1},$$

где k — номер структурного элемента (последний элемент $k = 1$, первый — $k = K$).

3. В качестве прототипа (приложение) взять диспетчерскую программу структурного элемента, выполненную на языке Паскаль. Первоначально провести в ней замену переменных:

```
X      P
C      F
A      D
DN=0
DV     PV
BV     BV
BN=0
```

4. Написать программу на языке Java и создать *.exe-файл.

Работа с генератором

1. Запустить созданный *.exe-файл, появится входной интерфейс (рис. 2.2).

2. Задать следующие параметры $K = 3$, $t = 2$, величины $M \times N$ не менее 2×2 . Величины P и D произвольны в пределах от 0 до 15.

3. Набор начинается с конечного элемента диспетчерской цепи до первого.

Vvedite chislo vremennyh intervalov I= .

Vvedite chislo reshaemyh zadach (elementov)#1 K=

N= Ввод числа прямых ограничений

M= Ввод числа основных ограничений

Vvedite plan

P[1]=

P[2]=

...

Vvedite matricu D

D[1,1]=

$D[1,2]=$

...

После набора для очередного k нажать кнопку NEXT.

Вариант А4

☐ Количество временных интервалов

☐ Количество решаемых задач

Размерность задачи:

N: M:

План X

1	

Матрица А

1	

Next

Рис. 2.2

Входной интерфейс

4. Результирующие данные помещаются в отдельные файлы вида OUT1.1, где первая цифра — номер интервала времени, вторая — номер структурного элемента. Вид файлов таков:

```
2<==N
2<==M
FP=20196.60
F
363.40 1592.20
PN
  00  0.00
PV
12.00 12.00
BN
  0.00  0.00
BV
162.00 39.00
D
  2.00 13.00
  5.00  2.00
P
12.00 12.00
Popt
  3.00 12.00
B=BV
162.00 39.00
```

Условные обозначения:

F — вектор для целевой функции;

FP — оптимальное значение целевой функции;

DN — нижние прямые ограничения;

DV — верхние прямые ограничения;

BN — нижние основные ограничения;

BV — верхние основные ограничения;

D — матрица основных ограничений;

P_{opt} — оптимальный план;

P — план;

B — вектор DP .

5. Просмотреть все файлы.

Содержание отчета

1. Титульный лист.

2. Задание.

3. Введение.

4. Математическое описание задачи.

5. Экранные формы интерфейсов пользователей.

6. Заключение.

3. Серверная связка

Введение

Работа 3. Серверная связка

Цель работы — проверить работоспособность серверной связки «сервер приложений — клиент».

В реализации технологии клиент-сервер выделяют сервер базы данных, сервер приложений и клиентов. Серверы часто выполняются в связке. Базы данных могут быть табличными и файловыми.

Для клиентов необходим универсальный язык программирования. К нему предъявляются следующие требования.

1. Расширяемость программной системы.
2. Переносимость.
3. Удобство использования и сопровождения программ с помощью соответствующего простого интерфейса.
4. Простота построения и освоения языка программирования.
5. Простота процесса программирования с минимумом ошибок.
6. Возможность формирования территориально распределенных систем с использованием различных вариантов технологий архитектуры клиент-сервер.
7. Возможность согласования работы с СУБД.
8. Наличие удобной среды выполнения.
9. Защита данных.

Перечисленным свойствам удовлетворяет язык Java с очень удобной средой выполнения IntelliJ IDEA.

Серверы баз данных и приложений выпускаются как согласованные программные продукты.

В связи с этим необходимо дополнительно рассмотреть вопросы согласования сервера приложений и языка программирования. Это сочетание назовем *серверной связкой*. Она состоит из клиента и сервера приложений с промежуточным слоем в виде сервлета.

Клиент находится на другом компьютере. Задача системы — передать текст из компьютера сервера приложений в клиент по его запросу.

Технология клиент-сервер отличается простой структурой, небольшими объемами передаваемых данных и слабой защитой данных и потому используется редко.

Объектная (удаленный объект RMI) и компонентная с промежуточными программными продуктами (продукт Java Beans) технологии отличаются повышенным объемом и безопасностью данных. RMI является развитием метода удаленной процедуры. Удаленный вариант обладает более широкими возможностями: в объекте может присутствовать несколько методов, что позволяет, в свою очередь, связать объект с другими объектами.

В то же время в них затруднены процедуры набора и проверки кода, а компонентная технология недостаточно детально проработана.

Более сложной структурой, дополненной сервлетами, или продуктами (Java Server Page (JSP), Enterprise Java Beans (EJB)), упрощением формирования кода и значительными объемами передаваемых хорошо защищенных данных, обладает сервисная технология.

Эта технология совместно с языками XML и протоколом http трансформируется в широко используемую веб-сервисную технологию, которая позволяет передавать значительные объемы данных при повышенной их безопасности. При использовании среды выполнения IntelliJ IDEA серьезно автоматизируется процедура построения кода с минимумом ошибок.

Широко распространенной является объектная технология с удаленным объектом (RMI), которая и обсуждается в данной лабораторной работе.

Описание работы

Сервер приложений и клиент расположены на разных компьютерах. В сервере приложения имеется текст (программа приложения), который по запросу клиента следует ему передать.

Необходимо программно построить связку и иллюстрировать ее в работе.

Коды файлов системы приведены в приложении. Для их создания использована среда разработки IntelliJ IDEA.

Программная реализация осуществляется на языке программирования Java.

Механизм распределенных объектов (Remote Method Invocation (RMI)) построен на обращении к методам объектов удаленной виртуальной машины Java.

Сервер (удаленная машина) создает объект, предоставляя его интерфейс, и ожидает обращения клиента к методам сервера. Клиент получает реализацию интерфейса как объект-заглушку и обращается к методам заглушки как к методам локального объекта. Взаимодействие заглушки с сервером осуществляется по специальному протоколу Java Remote Method Protocol.

Структура программной реализации показана на рисунке 3.1.

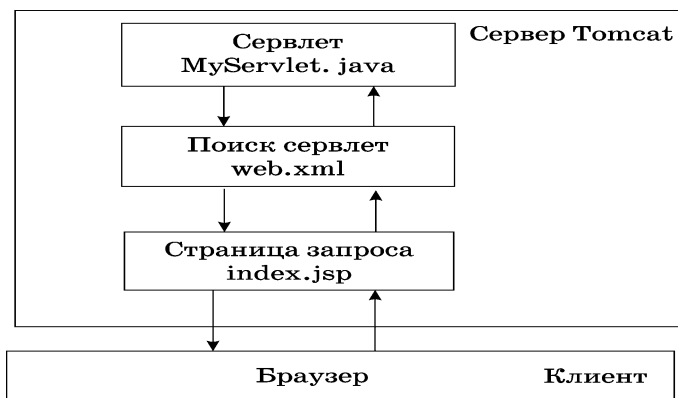


Рис. 3.1

Схема системы

Порядок выполнения работы

Создание системы

Для реализации проекта надо скачать на компьютер Java SE Development Kit 8 (JDK) комплект разработчика приложений на языке Java, включающий в себя компилятор Java (javac), стандартные библиотеки классов Java, примеры, документацию, различные утилиты и исполнительную систему Java (JRE). Дополнительно потребуется скачать IntelliJ IDEA Version: 2018 — интегрированную среду разработки программного обеспечения для многих языков программирования.

Запустить скачанный файл `jdk-8u181-windows-x64.exe`, для установки Java SE Development Kit 8 следовать инструкции.

Установить среду разработки IntelliJ IDEA Community Version: 2018, скачанный файл называется `ideaIC-2018.2.4.exe`. При первом запуске IntelliJ IDEA следует указать на отсутствие сохранённых настроек. Если настройки уже есть, то указать путь до них. Пролистать текст до конца и принять лицензионное соглашение. Возможно принять или отклонить предложение об отправке отчётов о работе программы в компанию JetBrains.

Приступить к формированию проекта и настройке среды. Создать проект, щелкнув на пункте `New` → `Project` в верхнем меню. В левой части выбрать `Maven`, а в правой сначала выбрать `Project SDK`, для чего нажать кнопку «new», и указать путь к SDK. Стандартный путь: `«C:\Program files\Java\jdk1.8.0_191»`. Поставить галочку напротив «Create from archetype» и выбрать архетип `maven-archetype-webapp` (рис. 3.2). Нажать «Next».

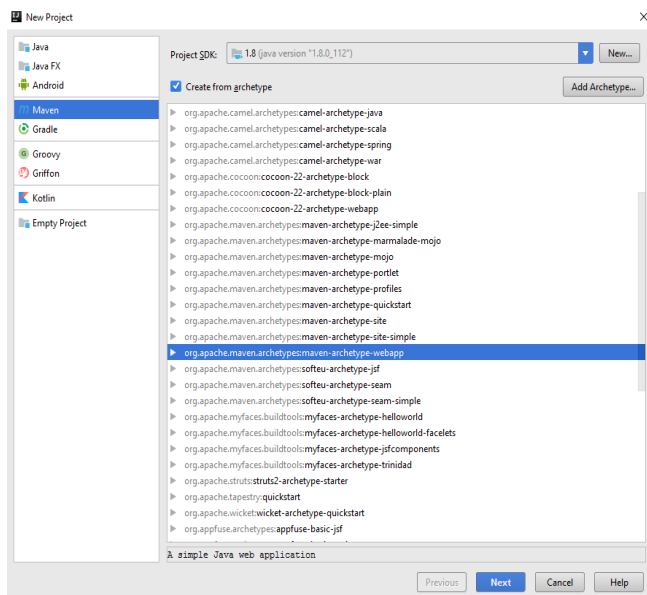


Рис. 3.2

Создание проекта Maven

Из следующих скриншотов (рис. 3.3, 3.4) видно, как задавать group id, artifact id, имя проекта и папку для него. Group id — уникальный идентификатор проекта, Обычно это имя java-пакета в соответствии с Java code conventions. Artifact id — имя JAR-файла, который заносится в репозиторий Maven, а в данном случае — в mywebapp. Нажать «Finish».

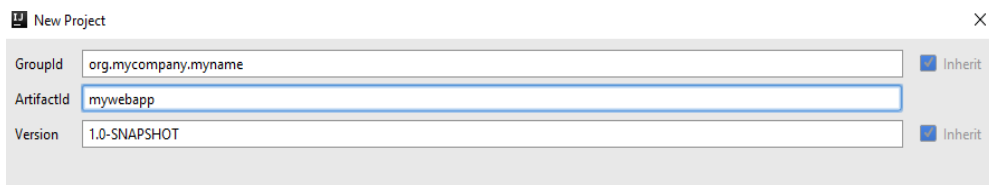


Рис. 3.3

Создание group id, artifact id

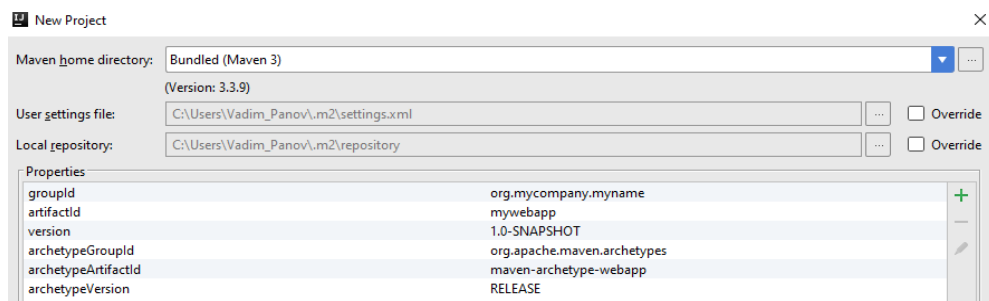


Рис. 3.4

Задание имени и директории проекта

После создания проекта появится стандартная структура папок, необходимая для веб-приложения: папка WEB-INF, файл web.xml и даже главная страница (рис. 3.5).

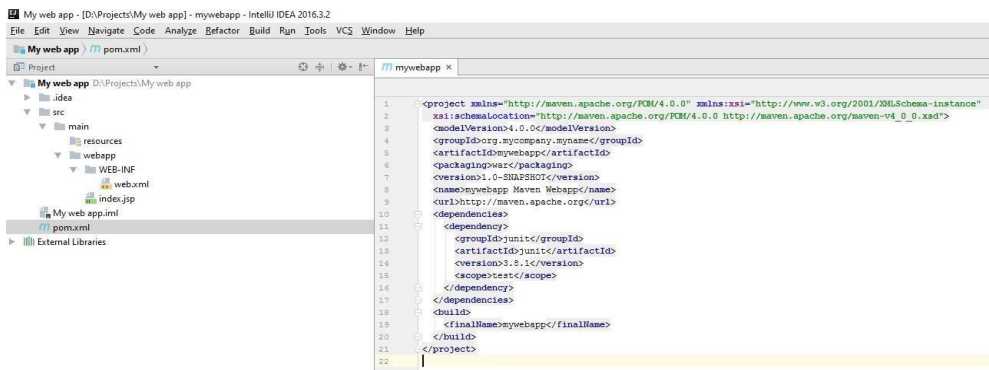


Рис. 3.5

Стандартная структура папок

Сборка проекта пройдет удачно, но надо указать адрес хранения — исходный код Java (рис. 3.6). Создать папку java по адресу My web app/src/main/, для чего правой кнопкой мыши нажать на папку main, затем «new» и, выбрав

directory, задать имя Java. Созданную папку отметим как "source root" (рис. 3.7).

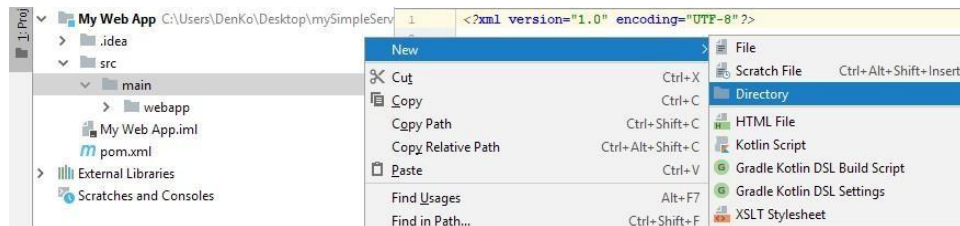


Рис. 3.6

Создание папки для исходников

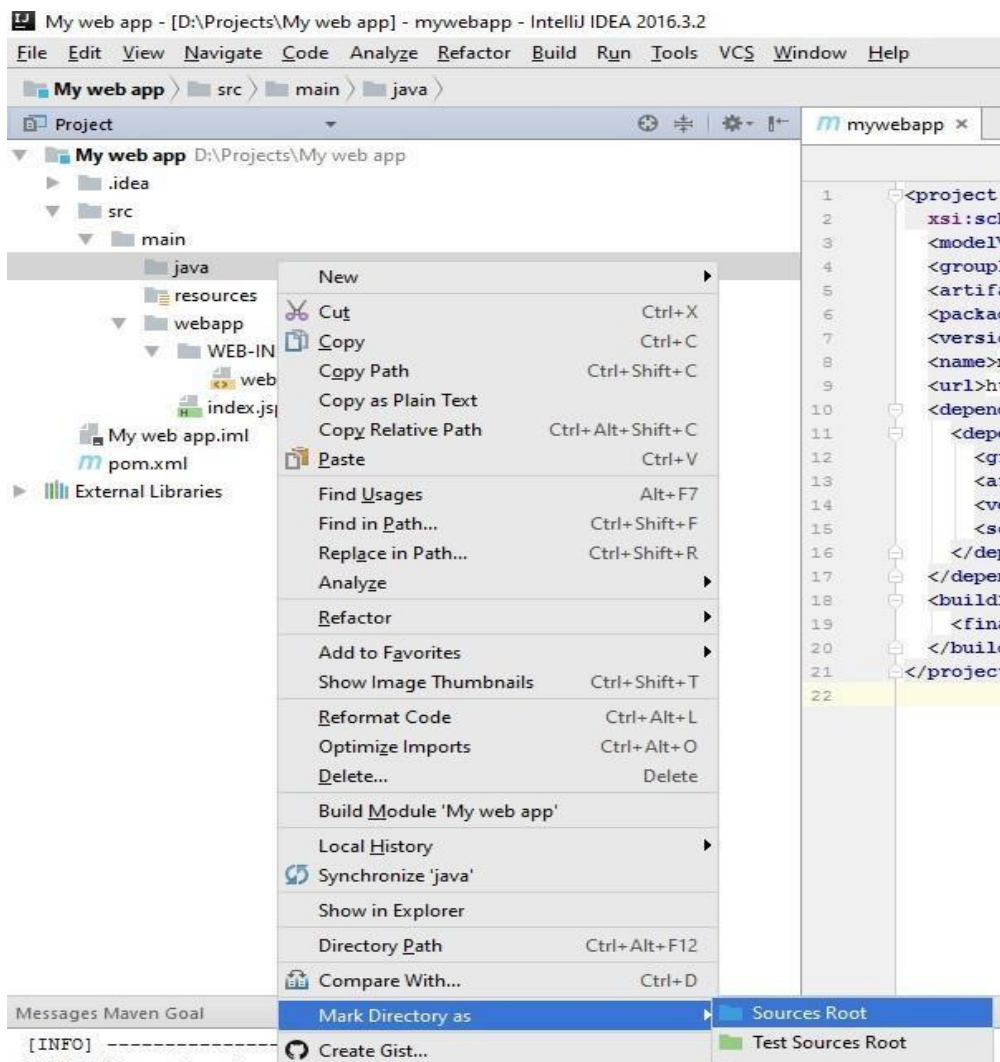


Рис. 3.7

Поставить метку

В этой папке хранится код сервлетов. Реализовать простой сервлет на базе класса `GenericServlet` и назвать его `MyServlet` (рис. 3.8), заменив весь его код на код из листинга 1 (приложение 3, коды программ). Дополнительно создать в этой папке файл класса `Java Class` и назвать его `TextPage*`, в этом файле будет храниться текст сервера для клиента. Пример кода для `TextPage` приведен в листинге 5 (приложение 3).

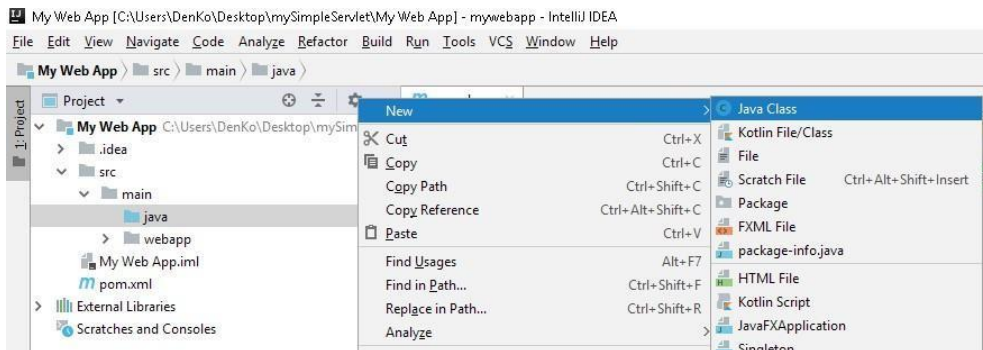


Рис. 3.8

Создание сервлета в папке Java

Далее добавить библиотеки Java EE при помощи Maven. Открыть `pom.xml` и привести его к виду листинга 2.

Возможно, чтобы собрать проект, надо добавить в `pom.xml` директивы, которые вытягивают с серверов Maven плагин `tomcat7-maven`. Воспользоваться этим плагином: нажать `Run` на главных вкладках, потом `Debug`, затем в появившемся окне `Edit Configuration` (рис. 3.9). Нажать «плюс» в верхнем левом углу и выбрать `Maven` (рис. 3.10), назвать ее `Tomcat`. В поле `Command line` написать `tomcat7:run`, нажать плюс внизу страницы. Наконец, добавить `clean` в список целей `Run Maven Goal` перед запуском (рис. 3.11) и нажать «Apply».

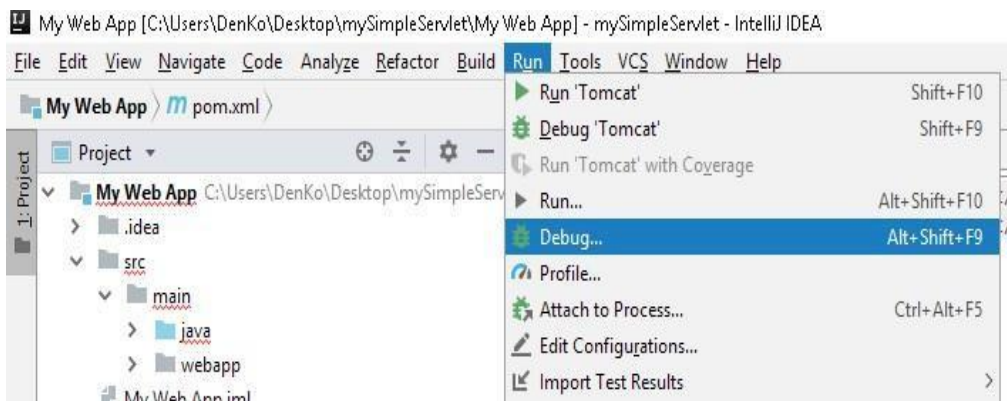


Рис. 3.9

Run Debug

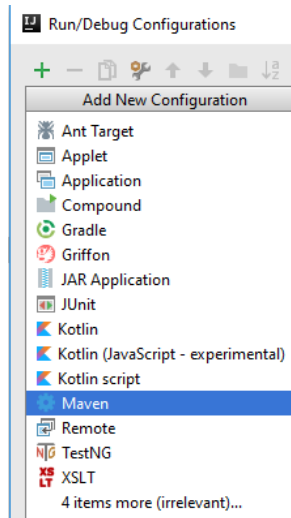


Рис. 3.10

Выбираем Maven

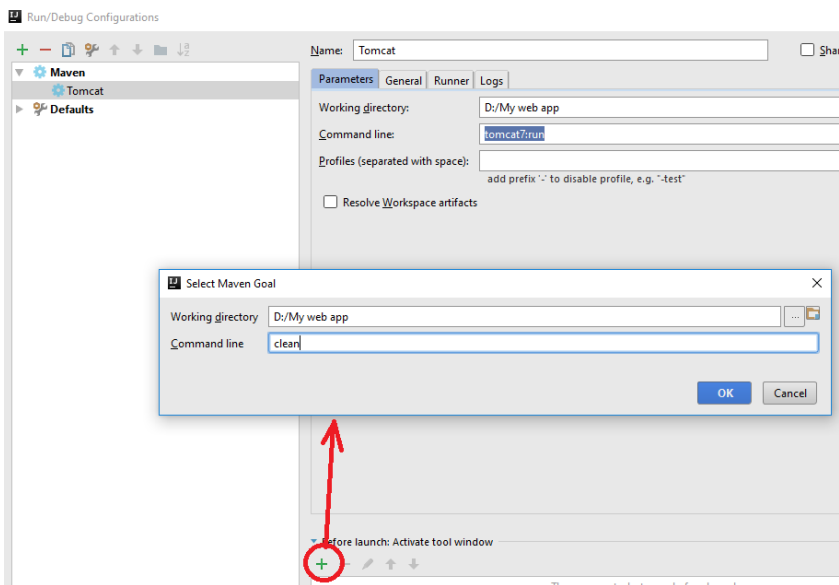


Рис. 3.11

Создаем конфигурацию запуска

Теперь можно нажать Shift+F10 (или кнопку Run на панели сверху) и увидеть в браузере страницу index.jsp по адресу <http://localhost:9091> (ссылка будет в окне вывода информации о сборке).

Чтобы обратиться к сервлету, надо дать Tomcat понять, где его искать и на какой адрес отправить. Для этого надо поменять два файла (листинги 3, 4; прил. 3): src/main/webapp/WEB_INF/web.xml и src/main/webapp/index.jsp.

Работа схемы

Выполнить команду `run` и перейти по ссылке `localhost:9091`, которая находится в окне вывода информации о сборке (рис. 3.12), или же непосредственно ввести её в браузер. Открывается окно сервлета с кнопкой «Request», при нажатии на которую осуществится переход на страницу просмотра текста.

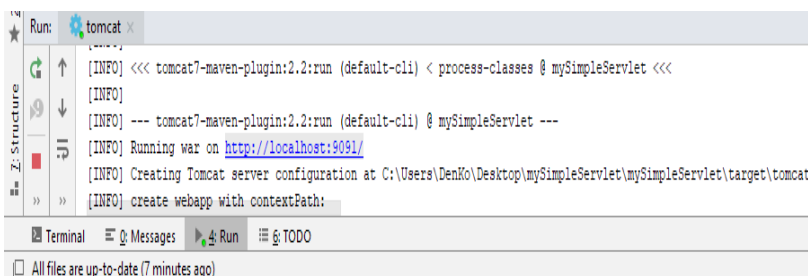


Рис. 3.12

Терминал среды со ссылкой на сервлет

Для клиента, компьютер которого подключен к локальной сети сервера, достаточно зайти в браузер и задать `ip`-адрес сервера, и порт осуществит переход на страницу сервлета (рис. 3.13).

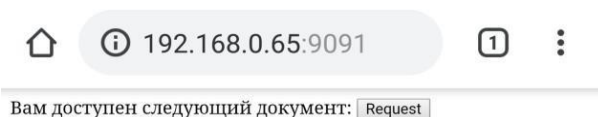


Рис. 3.13

Клиент подключается к сервлету

Нажав кнопку запроса «Request», клиент-пользователь увидит следующий текст в браузере, полученный из сервера (рис. 3.14).

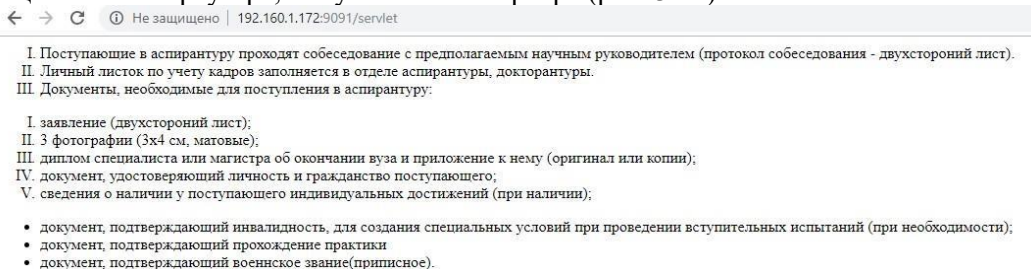


Рис. 3.14

Текст из сервера

Таким образом, серверная связка сработала. Коды программ приведены в приложении 3.

Содержание отчета

1. Титульный лист.
2. Задание.
3. Введение.
4. Описание задачи.
5. Экранная форма результата.
6. Заключение.

4. Система планирования «толстый» клиент-сервер

Введение

Работа 4. «Толстый» клиент с файловой БД

Цель работы — построение распределенной двухуровневой системы управления.

Распределенные (сетевые) системы в последнее время получают все более широкое распространение. В данной работе рассматривается система «сервер БД — сервер приложений — клиенты». В качестве СУБД используется SQLite.

Описание работы

Необходимо по запросу клиента передать ему содержимое таблицы БД из сервера.

Обобщенная программная структура представлена на рисунке 4.1, а детальная — на рисунке 4.2.

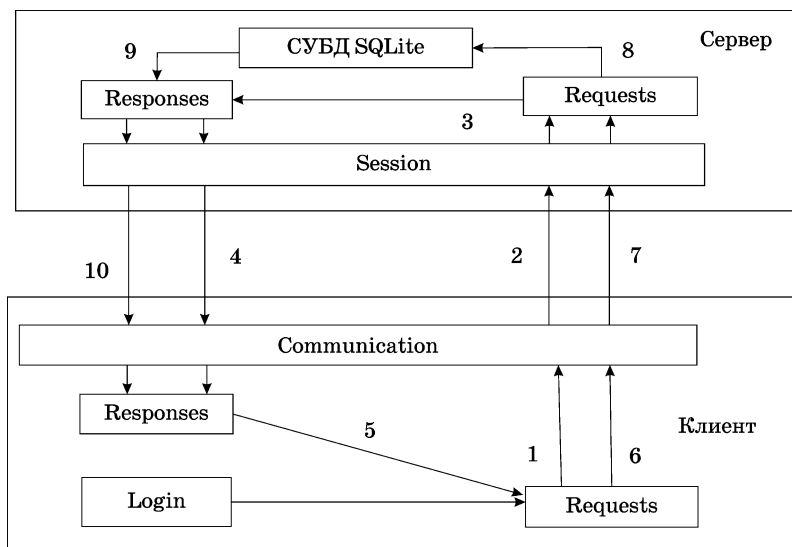


Рис. 4.1

Обобщенная схема системы:
1–4 — запрос и ответ на установление связи; 5–10 — запрос и ответ на выдачу данных.

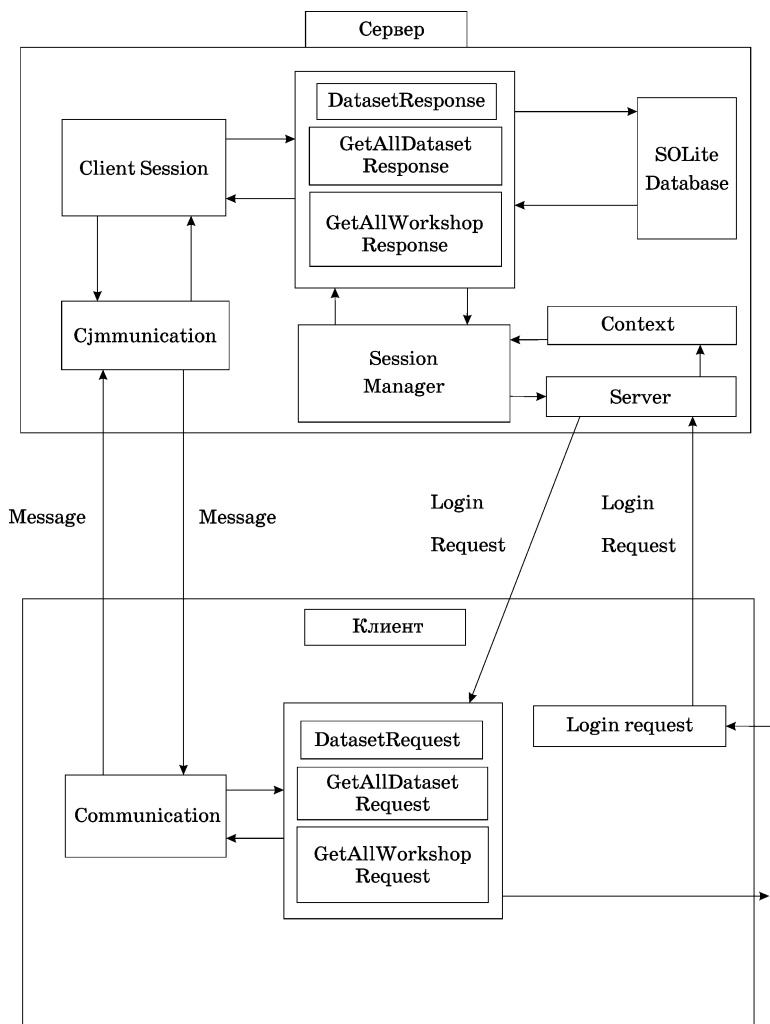


Рис. 4.2

Детальная программная структура

Файлы имеют следующее назначение.

Клиентская часть

Tcp.Core — основной пакет, содержащий клиентскую и серверную части.

Request — описывает необходимые данные для запроса, наследует интерфейс IMessage.

Response — описывает необходимые данные для ответа, наследует интерфейс IMessage.

Следующие три класса будут наследовать Serializable, что поможет передавать данные по программе в виде байт кода.

Dataset Object — содержит в себе объекты данных, т. е. матрицы, имена.

Login Object — объекты, необходимые для входа в систему, т. е. логин и пароль, роль.

Workshop Object — содержит данные, изменяемые в ходе работы.

Пакет Request выполняет следующие функции:

- Dataset Request — реализует поток данных запросов с применением пространства имен Dataset Object;
- Get All Dataset Request — заносит в запрос данные в массиве;
- Get All Workshop Request — заносит в запрос рабочие (измененные) данные в массиве;
- Login Request — запрос на вход в систему.

Пакет Response выполняет следующие функции:

- Dataset Response — реализует поток данных ответов с применением пространства имен Dataset Object;
- Get All Dataset Response — заносит в ответ данные в массиве;
- Get All Workshop Response — заносит в ответ все рабочие (измененные) данные в массив;
- Login Response — ответ на вход в систему.

Пакет Communication реализует следующие функции:

- Message Writer — инструмент для написания сообщений;
- Message Reader — инструмент для чтения созданных сообщений;
- Message Factory — класс, что преобразует, структурирует данные на основе запросов и ответов, формируя их в один пакет сообщений для удобной работы системы (преобразует данные в байт вид).

На этом пакет Tcp.core готов.

Пакет Client выполняет следующую функцию:

- реализует всю клиентскую базу, содержит метод входа в систему, получения и отправки данных под видом сообщений.

Пакет Gui выполняет следующую функцию:

- содержит FXML-файлы для описания интерфейса программы.

Пакет Controllers выполняет следующую функцию:

- осуществляет правильный процесс объединения элементов интерфейса и программных методов для обеспечения полного функционала программы.

Серверная часть

Серверная часть также содержит пакет Tcp.core.

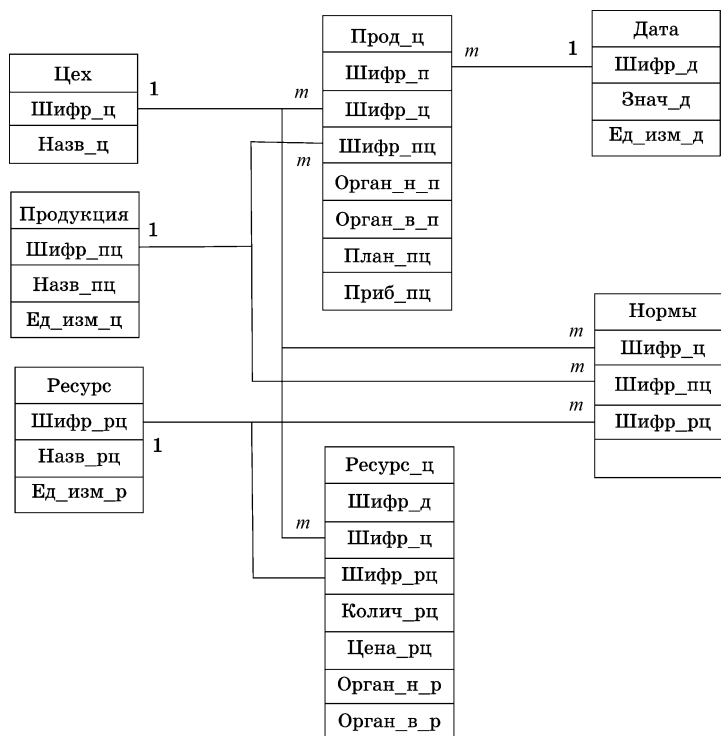
Tcp.server — пакет — основа серверной части, выполняет следующие функции:

- ClientSession — реализует сессионный механизм пользователя и организует работу сервера, т. е. обмен сообщениями, хранение информации;
- Context — реализует механизм соединения;
- Server — сервер, содержит серверную информацию, отображает подключившихся;
- ServerLauncher — механизм запуска сервера;
- SessionManager — создает сессию.

Для сервера формируются программы slp-client, slp-server, slp.tpc.server, SQLite. Дополнительно необходимы программы конфигурации запуска, сборки итогового файла и приложения, соединений связи БД с java-программами.

Порядок выполнения работы

Установка программного продукта. Все программные модули скомпонованы в две папки server и client. Папки server и client устанавливаются на соответствующих компьютерах в доступных для пользователя местах. Например, таким местом может быть Рабочий стол или Documents. В данном случае будет находиться на Рабочем столе. Папка client будет выступать клиентской машиной, папка server будет выступать серверной машиной. Обе машины должны находиться в одной сети и иметь разные ip-адреса. При желании можно использовать виртуальную локальную сеть, если получится настроить. Обмен между сервером и клиентом будет идти по протоколу tcp/ip.



После установки и запуска SQLite нажать «Новая база данных», выбрать корневой каталог (например, slp-server), ввести название db (рис. 4.4). В типе файла выбрать «Все файлы» и самостоятельно дописать файлу разрешение «s3db».

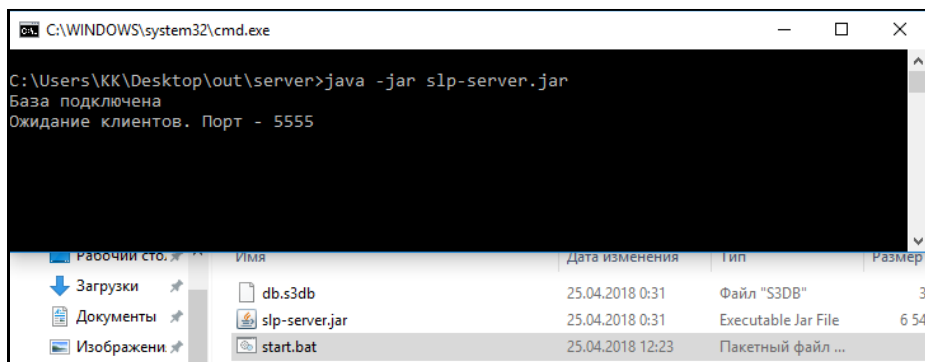


Рис. 4.4

Программный состав системы

Нажать «Сохранить» и закрыть появившееся окно создания таблицы. Перейти на вкладку SQL и вставить код.

```
BEGIN TRANSACTION;
DROP TABLE IF EXISTS `workshops`;
CREATE TABLE IF NOT EXISTS `workshops` (
  `id` INTEGER PRIMARY KEY AUTOINCREMENT,
  `name` INTEGER
);
DROP TABLE IF EXISTS `users`;
CREATE TABLE IF NOT EXISTS `users` (
  `id` INTEGER PRIMARY KEY AUTOINCREMENT,
  `name` TEXT NOT NULL UNIQUE,
  `password` TEXT NOT NULL,
  `role` INTEGER NOT NULL DEFAULT 0,
  `workshop` INTEGER,
  FOREIGN KEY(`workshop`) REFERENCES `workshops`(`id`)
);
INSERT INTO `users` VALUES (1,'admin','123',2,NULL);
DROP TABLE IF EXISTS `datasets`;
CREATE TABLE IF NOT EXISTS `datasets` (
  `id` INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
  `name` TEXT,
  `M` INTEGER,
  `N` INTEGER,
  `data` TEXT,
  `workshop` INTEGER,
  FOREIGN KEY(`workshop`) REFERENCES `workshops`(`id`)
);
INSERT INTO `datasets` VALUES (1,'test1',3,2,'1;2;3;4;5;6',NULL);
INSERT INTO `datasets` VALUES (2,'test2',NULL,4,NULL,NULL);
COMMIT;
```

Нажать Старт (F5), перейти на вкладку «Структура БД». Видно, что наша база заполнена.

В сервере необходимо открыть порт 5555 для создания сетевой связи. Ввести IP-адрес сервера

В командной строке WindowsCMDIP-Config ввести IP-адрес.

После запуска клиента ввести его IP-адрес.

Работа с системой

Установить пакет server на сервере, а пакет client — на клиенте.

Для запуска сервера необходимо запустить файл «start.bat» из папки «server» (рис. 4.5).

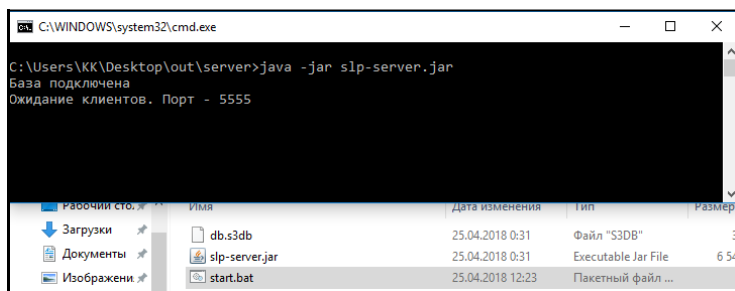


Рис. 4.5

Окно запуска сервера

Для запуска приложения следует перейти в папку «client» и запустить файл «slp_client.jar». Появится окошко авторизации пользователя (рис. 4.6).

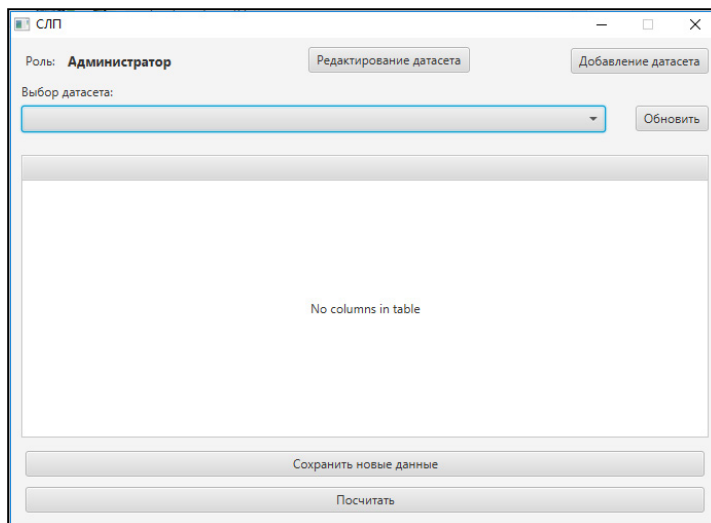


Рис. 4.6

Окно авторизации

В поле «Адрес сервера» ввести адрес машины-сервера: машины клиента и сервера находятся в одной сети.

Если вход осуществляется с машины, на которой расположен сервер, то поле «Адрес сервера» оставить пустым. В поле «Логин» и «Пароль» вводятся учетные записи пользователя, после чего открывается основной интерфейс программы (рис 4.7).

Рис. 4.7

Основной интерфейс программы

Учетные записи «Логин — пароль» имеют вид: администратор admin — 123; диспетчер — operator — 321; начальники цехов — foreman1 — 456, foreman2 — 678, foreman3 — 890.

Только администратор может добавлять новые таблицы в базу данных. Он может создать пустую таблицу или сразу заполнить её данными с помощью генератора (рис. 4.8).

Рис. 4.8

Создание таблицы

Администратор может изменять существующий набор данных (таблицу) по нажатию кнопки «Редактирование датасета» (рис. 4.9).

Рис. 4.9

Изменение таблицы

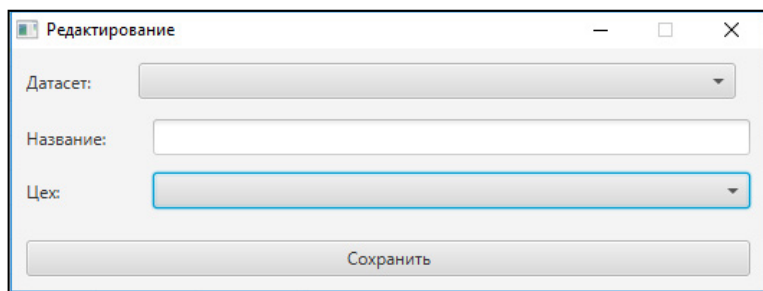


Рис. 4.10

Закрепление задач за начальником цеха

С помощью данного функционала он может назначить уникальное имя таблице и закрепить её за одним из начальников цехов (рис. 4.10).

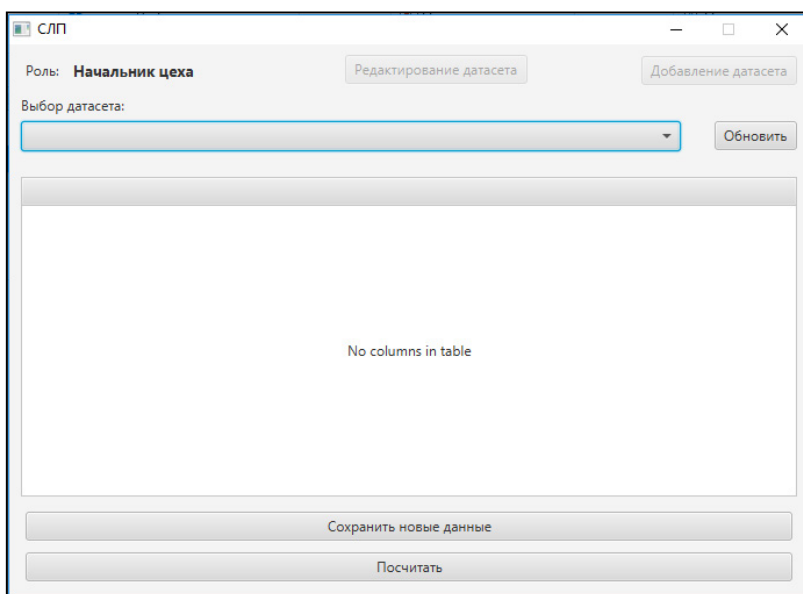


Рис. 4.11

Введение таблицы «Инструкция диспетчеру»

Администратор может самостоятельно осуществить введение таблицы или просто сохранить её в системе для дальнейшего решения (рис. 4.11).

Диспетчер (оператор) является связующим (промежуточным) звеном между исполнителями (начальниками цехов) и руководителем (администратором). Его полномочия сводятся только к закреплению конкретных таблиц за начальниками цехов. Осуществить это можно нажатием на кнопку «Редактирование датасета».

Инструкция начальнику цеха

Начальник цеха может работать только с назначенными ему таблицами. В его полномочия входят: расчёт закрепленных за ним таблиц, внесение в них

изменений (только данные), сохранение измененных таблиц под новым именем и дальнейшее их использование (рис. 4.12).

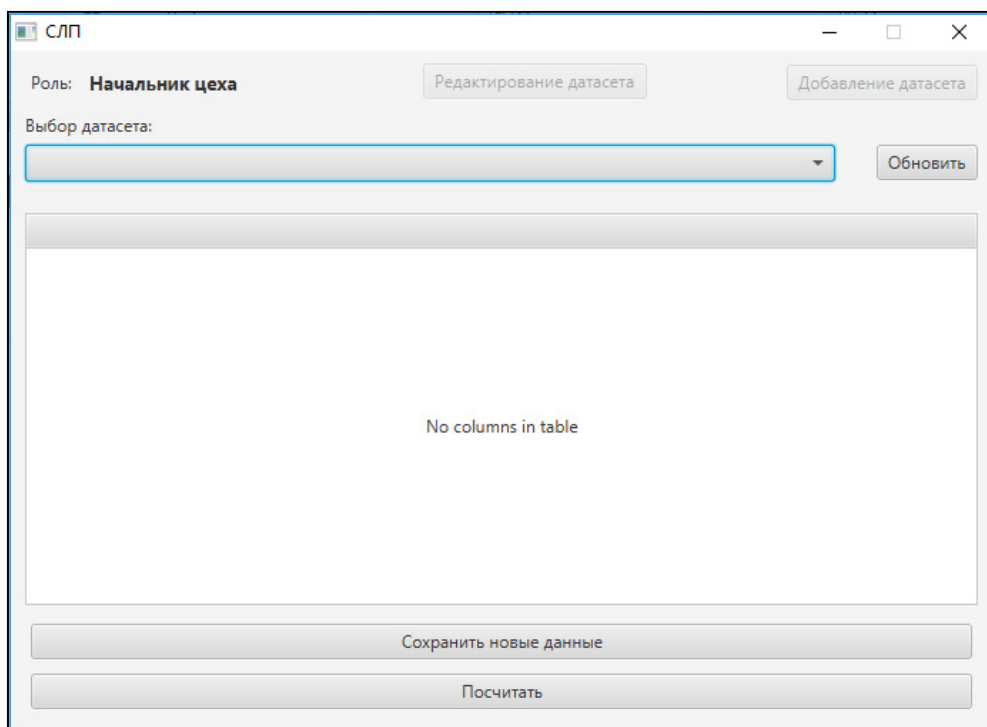


Рис. 4.12

Интерфейс начальника цеха

Содержание отчета

1. Титульный лист.
2. Задание.
3. Введение.
4. Описание задачи.
5. Экранная форма результата.
6. Заключение.

Коды программ связей локального варианта распределенной системы (ЛР1)

Серверные программы

```
SET SQL DIALECT 3;
```

Создание файла БД

```
/* CREATE DATABASE
```

```
'C:\Users\А×\Desktop\ÔĚÝØ1\Ñîîâîâîâ_Íâîâñòüâââ\SKL.GDB'PAGE_SIZE 4096
```

```
DEFAULT CHARACTER SET WIN1251 */
```

```
/* External Function declarations */
```

Внешние процедуры

```
DECLARE EXTERNAL
```

```
FUNCTION IIFINTEGER,
```

```
INTEGER, INTEGER RETURNS
```

```
INTEGER BY VALUE
```

```
ENTRY_POINT 'fn_iif' MODULE_NAME 'rfunc';
```

```
DECLARE EXTERNAL FUNCTION
```

```
MODINTEGER, INTEGER
```

```
RETURNS DOUBLE PRECISION BY VALUE
```

```
ENTRY_POINT 'IB_UDF_mod' MODULE_NAME 'ib_udf';
```

Входные таблицы

```
/* Table: DETALI, Owner:
```

```
SYSDBA */CREATE TABLE
```

```
"DETALI"
```

```
(
```

```
"SHIFR_DET" CHAR(2) CHARACTER SET WIN1251 NOT NULL,
```

```
"NAZV_DET" CHAR(20) CHARACTER SET
```

```
WIN1251,"ED_Izm" CHAR(2) CHARACTER SET
```

```
WIN1251, PRIMARY KEY ("SHIFR_DET")
```

```
);
```

```
/* Table: IZDEL, Owner:
```

```
SYSDBA */CREATE TABLE
```

```
"IZDEL"
```

```
(
```

```
"SHIFR_Izd" CHAR(2) CHARACTER SET WIN1251 NOT
```

```
NULL,"NAZV" CHAR(20) CHARACTER SET WIN1251,
```

```
"ED_Izm" CHAR(3) CHARACTER SET WIN1251,
```

```
PRIMARY KEY ("SHIFR_Izd")
```

```
);
```

```
/* Table: MATER, Owner:
```

```
SYSDBA */CREATE TABLE
```

```

"MATER"
(
  "SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251 NOT NULL,
  "NAZV_MAT" CHAR(20) CHARACTER SET
  WIN1251,"ED_IZM" CHAR(2) CHARACTER SET
  WIN1251, "CENA" INTEGER,
  "INTERV_POST"
  INTEGER, "T_VIPOLN"
  INTEGER,
  "RASN" INTEGER,
  PRIMARY KEY
  ("SHIFR_MAT")
);

```

```

/* Table: NORMI, Owner: */
SYSDBACREATE TABLE
"NORMI"
(
  "SHIFR_MAT" CHAR(2) CHARACTER SET
  WIN1251,"SHIFR_DET" CHAR(2)
  CHARACTER SET WIN1251,"ED_IZM" CHAR(5)
  CHARACTER SET WIN1251, "NORM_RASH"
  NUMERIC(15, 2) NOT NULL
);

```

```

/* Table: PLAN_VIPUSK1, Owner: SYSDBA */
CREATE TABLE "PLAN_VIPUSK1"
(
  "DATA" INTEGER NOT NULL,
  "SHIFR_IZD" CHAR(2) CHARACTER SET WIN1251 NOT
  NULL,"KOL" INTEGER NOT NULL,
  PRIMARY KEY ("DATA", "SHIFR_IZD")
);

```

```

/* Table: VHOD, Owner:
SYSDBA */CREATE TABLE
"VHOD"
(
  "SHIFR_DET" CHAR(2) CHARACTER SET WIN1251 NOT
  NULL, "SHIFR_IZD" CHAR(2) CHARACTER SET WIN1251
  NOT NULL, "ED_IZM" CHAR(6) CHARACTER SET
  WIN1251,
  "KOL" INTEGER NOT NULL
);

```

Выходная таблица

```

/* Table: ZAPAS, Owner: SYSDBA

```

```

*/CREATE TABLE "ZAPAS"
(
  "SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251 NOT
  NULL,"DATA" INTEGER,
  "OLD_ZAPAS" DOUBLE PRECISION
);

```

```

/* Stored procedures */

```

Среднее

```

CREATE PROCEDURE

```

```

"AVERADD"RETURNS

```

```

(
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_AVERAGE" DOUBLE PRECISION
)

```

```

AS

```

```

BEGIN EXIT; END ^

```

Совмещение

```

CREATE PROCEDURE

```

```

"POSPOT2"RETURNS

```

```

(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_POSTAV" DOUBLE
  PRECISION,"OUT_POTREB"
  DOUBLE PRECISION
)

```

```

AS

```

```

BEGIN EXIT; END ^

```

Новый запас

```

CREATE PROCEDURE

```

```

"POSPOT3"RETURNS

```

```

(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "NEW_ZAPAS" DOUBLE PRECISION,
  "DATA1" INTEGER
)

```

Поставка

```

CREATE PROCEDURE

```

```

"POST"RETURNS

```

```

(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_POSTAV" DOUBLE PRECISION
)

```

```

AS
BEGIN EXIT; END ^

CREATE PROCEDURE
"POTREBN1"RETURNS
(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET
WIN1251, "OUT_NAZV_MAT" CHAR(20)
CHARACTER SET WIN1251, "OUT_POTREB"
DOUBLE PRECISION
)
AS
BEGIN EXIT; END ^
Модуль
CREATE PROCEDURE
"POTREBN3"RETURNS
(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_RASN"
INTEGER,
  "OUT_MODUL"
INTEGER
)
AS
BEGIN EXIT; END ^
МОДУЛЬ1
CREATE PROCEDURE
"POTREBN4"RETURNS
(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_MODUL1" INTEGER
)
AS
BEGIN EXIT; END ^
Заказ
CREATE PROCEDURE
"ZAKAZ3"RETURNS
(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_ZAKAZ" DOUBLE PRECISION

```

```

ALTER PROCEDURE
"AVERADD"RETURNS
(
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_AVERAGE" DOUBLE PRECISION
)
AS
begin
for select POTREBN1.OUT_SHIFR_MAT, AVG(OUT_POTREB*4)
from POTREBN1
GROUP BY
POTREBN1.OUT_SHIFR_MAT INTO
:OUT_SHIFR_MAT, :OUT_AVERAGEDO
SUSPEND;
end
^

```

```

ALTER PROCEDURE
"POSPOT2"RETURNS
(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_POSTAV" DOUBLE PRECISION,
  "OUT_POTREB" DOUBLE PRECISION
)
AS
begin
for select POST.OUT_DATA, POST.OUT_SHIFR_MAT,
POST.OUT_POSTAV, (SELECT OUT_POTREB FROM
POTREBN1where (POTREBN1.OUT_DATA =POST.OUT_DATA)
AND (POTREBN1.OUT_SHIFR_MAT
=POST.OUT_SHIFR_MAT))
from POST
INTO :OUT_DATA, :OUT_SHIFR_MAT, :OUT_POSTAV, :OUT_POTREB
DO
SUSPEND;
end
^

```

```

ALTER PROCEDURE
"POSPOT3"RETURNS
(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "NEW_ZAPAS" DOUBLE PRECISION,
  "DATA1" INTEGER
)

```

```

AS
DECLARE VARIABLE N INTEGER;
begin
N=1;
WHILE (N<24) DO
begin
for select ZAPAS.DATA, ZAPAS.SHIFR_MAT,
(ZAPAS.OLD_ZAPAS+POSPOT2.OUT_POSTAV-
POSPOT2.OUT_POTREB), (ZAPAS.DATA+1)
FROM POSPOT2, ZAPAS
where (POSPOT2.OUT_DATA=ZAPAS.DATA)
AND
(POSPOT2.OUT_SHIFR_MAT=ZAPAS.SHIFR_M
AT)AND (POSPOT2.OUT_DATA=:N)
INTO :OUT_DATA, :OUT_SHIFR_MAT, :NEW_ZAPAS, :DATA1
DO
begin
update zapas
set DATA=:DATA1, SHIFR_MAT=:OUT_SHIFR_MAT,
OLD_ZAPAS=:NEW_ZAPAS;
end
end
end
^

```

```

ALTER PROCEDURE
"POST"RETURNS
(
"OUT_DATA" INTEGER,
"OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
"OUT_POSTAV" DOUBLE PRECISION
)
AS
begin
for select POTREBN3.OUT_DATA,
POTREBN3.OUT_SHIFR_MAT,
IIF(POTREBN3.OUT_MODUL, 0,
AVERADD.OUT_AVERAGE) from POTREBN3, AVERADD
where
POTREBN3.OUT_SHIFR_MAT
=AVERADD.OUT_SHIFR_MAT
ORDER BY
POTREBN3.OUT_DATA, POTREBN3.OUT_SHIFR_MAT INTO
:OUT_DATA, :OUT_SHIFR_MAT, :OUT_POSTAV
DO
SUSPEND;
end
^

```

```

ALTER PROCEDURE
"POTREBN1"RETURNS
(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_NAZV_MAT" CHAR(20)
  CHARACTER SET WIN1251, "OUT_POTREB"
  DOUBLE PRECISION
)
AS
begin
for select PLAN_VIPUSK1.DATA, MATER.SHIFR_MAT, MATER.NAZV_MAT,
SUM(NORMI.NORM_RASH*VHOD.KOL*PLAN_VIPUSK1.KOL)
from MATER, NORMI, VHOD, PLAN_VIPUSK1, DETALI, IZDEL
where (MATER.SHIFR_MAT= NORMI.SHIFR_MAT) AND (DE-
TALI.SHIFR_DET=NORMI.SHIFR_DET)
AND (DETALI.SHIFR_DET=VHOD.SHIFR_DET) AND
(IZDEL.SHIFR_Izd=VHOD.SHIFR_Izd) AND
(IZDEL.SHIFR_Izd=PLAN_VIPUSK1.SHIFR_Izd)
GROUP BY PLAN_VIPUSK1.DATA, MATER.SHIFR_MAT,
MATER.NAZV_MAT INTO :OUT_DATA, :OUT_SHIFR_MAT,
:OUT_NAZV_MAT, :OUT_POTREB DO
SUSPEND;
end
^

```

```

ALTER PROCEDURE
"POTREBN3"RETURNS
(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_RASN" INTEGER,
  "OUT_MODUL" INTEGER
)
AS
begin
for select POTREBN1.OUT_DATA, POTREBN1.OUT_SHIFR_MAT,
MATER.RASN,
MOD(POTREBN1.OUT_DATA, MATER.INTERV_POST)
from POTREBN1, MATER
where MATER.SHIFR_MAT= POTREBN1.OUT_SHIFR_MAT
INTO :OUT_DATA, :OUT_SHIFR_MAT,
:OUT_RASN,:OUT_MODUL DO
SUSPEND;
end
^

```

```

ALTER PROCEDURE
"POTREBN4"RETURNS
(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_MODUL1" INTEGER
)
AS
begin
for select POTREBN1.OUT_DATA, POTREBN1.OUT_SHIFR_MAT,
MOD((POTREBN1.OUT_DATA+MATER.T_VIPOLN),
MATER.INTERV_POST)from POTREBN1, MATER
where MATER.SHIFR_MAT=
POTREBN1.OUT_SHIFR_MATINTO :OUT_DATA,
:OUT_SHIFR_MAT,:OUT_MODUL1
DO
SUSPEND;
end
^

```

```

ALTER PROCEDURE
"ZAKAZ3"RETURNS
(
  "OUT_DATA" INTEGER,
  "OUT_SHIFR_MAT" CHAR(2) CHARACTER SET WIN1251,
  "OUT_ZAKAZ" DOUBLE PRECISION
)
AS
begin
for select POTREBN4.OUT_DATA,
POTREBN4.OUT_SHIFR_MAT,
IIF(POTREBN4.OUT_MODUL1, 0,
AVERADD.OUT_AVERAGE) from POTREBN4, AVERADD
where
POTREBN4.OUT_SHIFR_MAT
=AVERADD.OUT_SHIFR_MAT
ORDER BY
POTREBN4.OUT_DATA, POTREBN4.OUT_SHIFR_MAT INTO
:OUT_DATA, :OUT_SHIFR_MAT, :OUT_ZAKAZ
DO
SUSPEND;
end
^

```

```

SET TERM ; ^
COMMIT WORK;
SET AUTODDL
ON;SET TERM ^;

```

/* Triggers only will work for SQL triggers */

Триггеры связей

```
CREATE TRIGGER "IZM_SHIFR_DET_NORM" FOR "DETALI"  
ACTIVE BEFORE UPDATE POSITION 0
```

```
as  
begin  
if(old.SHIFR_DET<>new.SHIFR_DET) then  
update NORMI  
set SHIFR_DET=new.SHIFR_DET  
where  
SHIFR_DET=old.SHIFR_DET;end  
^
```

```
CREATE TRIGGER "IZM_SHIFR_DET_VHOD" FOR "DETALI"  
ACTIVE BEFORE UPDATE POSITION 0
```

```
as  
begin  
if(old.SHIFR_DET<>new.SHIFR_DET) then  
update VHOD  
set SHIFR_DET=new.SHIFR_DET  
where  
SHIFR_DET=old.SHIFR_DET;end  
^
```

```
CREATE TRIGGER "UD_SHIFR_DET_NORM1" FOR "DETALI"  
ACTIVE AFTER DELETE POSITION 0
```

```
as  
begin  
delete from NORMI  
where NORMI.SHIFR_DET=DETALI.SHIFR_DET;  
end  
^
```

```
CREATE TRIGGER "UD_SHIFR_DET_VHOD1" FOR "DETALI"  
ACTIVE AFTER DELETE POSITION 0
```

```
as  
begin  
delete from VHOD  
where VHOD.SHIFR_DET=DETALI.SHIFR_DET;  
end  
^
```

```
CREATE TRIGGER "IZM_SHIFR_Izd_PLAN" FOR "IZDEL"  
ACTIVE BEFORE UPDATE POSITION 0
```

```
as  
begin
```

```
if(old.SHIFR_Iزد<>new.SHIFR_Iزد) then
update PLAN_VIPUSK
set SHIFR_Iزد=new.SHIFR_Iزد
where
SHIFR_Iزد=old.SHIFR_Iزد;end
^
```

```
CREATE TRIGGER "IZM_SHIFR_Iزد_VHOD" FOR "IZDEL"
ACTIVE BEFORE UPDATE POSITION 0
as
begin
if(old.SHIFR_Iزد<>new.SHIFR_Iزد) then
update VHOD
set SHIFR_Iزد=new.SHIFR_Iزد
where
SHIFR_Iزد=old.SHIFR_Iزد;end
^
```

```
CREATE TRIGGER "UD_SHIFR_Iزد_PLAN1" FOR "IZDEL"
ACTIVE AFTER DELETE POSITION 0
as
begin
delete from PLAN_VIPUSK
where PLAN_VIPUSK.SHIFR_Iزد=IZDEL.SHIFR_Iزد;
end
^
```

```
CREATE TRIGGER "UD_SHIFR_Iزد_VHOD" FOR "IZDEL"
ACTIVE AFTER DELETE POSITION 0
as
begin
delete from VHOD
where VHOD.SHIFR_Iزد=IZDEL.SHIFR_Iزد;
end
^
```

```
CREATE TRIGGER "IZM_POTREBNOS" FOR "MATER"
ACTIVE BEFORE UPDATE POSITION 0
as
begin
update POTREBNOS
set
POTREBNOS.INTERV_POST=MATER.INTERV_POST
where
MATER.SHIFR_MAT=POTREBNOS.SHIFR_MAT;
end
^
```

```
CREATE TRIGGER "IZM_SHIFR_MAT_NORM" FOR "MATER"
ACTIVE BEFORE UPDATE POSITION 0
```

```
as
begin
if(old.SHIFR_MAT<>new.SHIFR_MAT) then
update NORMI
set SHIFR_MAT=new.SHIFR_MAT
where
SHIFR_MAT=old.SHIFR_MAT;end
^
```

```
CREATE TRIGGER "UD_SHIFR_MAT_NORM1" FOR "MATER"
ACTIVE AFTER DELETE POSITION 0
```

```
as
begin
delete from NORMI
where NORMI.SHIFR_MAT=MATER.SHIFR_MAT;
end
^
```

```
COMMIT WORK
```

```
^SET TERM ;^
```

Программы клиента

```
unit Unit1;
```

```
interface
```

```
uses
```

```
Windows, Messages, SysUtils, Variants, Classes, Graphics, Controls,
Forms, Dialogs, Menus, DB, IBCustomDataSet, IBQuery, IBDatabase,
Grids, DBGrids, StdCtrls;
```

```
type
```

```
TForm1 = class(TForm)
```

```
MainMenu1:
```

```
TMainMenu;N1:
```

```
TMenuItem;
```

```
N2: TMenuItem;
```

```
N3:
```

```
TMenuItem;
```

```
N4:
```

```
TMenuItem;
```

```
DataSource1: TDataSource;
```

```
DBGrid1: TDBGrid;
```

```
IBDatabase1: TIBDatabase;
IBTransaction1:
TIBTransaction;IBQuery1:
TIBQuery;
Label1: TLabel;
N5:
TMenuItem;
N6:
TMenuItem;
N7:
TMenuItem;
N8:
TMenuItem;
N9:
TMenuItem;
N10:
TMenuItem;
N11:
TMenuItem;
N12:
TMenuItem;
N13:
TMenuItem;
N14:
TMenuItem;
N15:
TMenuItem;
procedure N4Click(Sender: TObject);
procedure N5Click(Sender: TObject);
procedure FormCreate(Sender: TObject);
procedure N6Click(Sender: TObject);
procedure N7Click(Sender: TObject);
procedure N8Click(Sender: TObject);
procedure N9Click(Sender: TObject);
procedure N14Click(Sender: TObject);
procedure N15Click(Sender: TObject);
procedure N10Click(Sender: TObject);
procedure N11Click(Sender: TObject);
procedure N12Click(Sender: TObject);
procedure N13Click(Sender: TObject);
private
{ Private declarations
}public
{ Public declarations
}end;
```

var

Form1: TForm1;

implementation

{ \$R *.dfm }

procedure TForm1.N4Click(Sender: TObject);

//Таблица Материалы

{

DBGrid1=>Visible=True;

Label1=>Caption='Материалы';

IBQuery1=>Close;

IBQuery1=>SQL=>Clear;

IBQuery1=>SQL=>Add('select * from MATER');

IBQuery1=>Open;

};

procedure TForm1.N5Click(Sender: TObject);

//Таблица Детали

{

DBGrid1=>Visible=True;

Label1=>Caption='Детали';

IBQuery1=>Close;

IBQuery1=>SQL=>Clear;

IBQuery1=>SQL=>Add('select * from DETALI');

IBQuery1=>Open;

};

procedure TForm1.FormCreate(Sender: TObject);

{

DBGrid1=>Visible=False;

};

procedure TForm1.N6Click(Sender: TObject);

//Таблица Изделие

{

DBGrid1=>Visible=True;

Label1=>Caption='Изделие';

IBQuery1=>Close;

IBQuery1=>SQL=>Clear;

IBQuery1=>SQL=>Add('select * from IZDEL');

IBQuery1=>Open;

};

procedure TForm1.N7Click(Sender: TObject);

```
//Таблица План
{
DBGrid1=>Visible=True
; Label1=>Caption='План';
IBQuery1=>Close;
IBQuery1=>SQL=>Clea
r;
IBQuery1=>SQL=>Add('select * from PLAN_VIPUSK1');
IBQuery1=>Open;
};
```

```
procedure TForm1.N8Click(Sender: TObject);
//Таблица Нормы
{
DBGrid1=>Visible=True;
Label1=>Caption='Нормы';
IBQuery1=>Close;
IBQuery1=>SQL=>Clear;
IBQuery1=>SQL=>Add('select * from NORMI');
IBQuery1=>Open;
};
```

```
procedure TForm1.N9Click(Sender: TObject);
//Таблица Входимость
{
DBGrid1=>Visible=True;
Label1=>Caption= 'Входимость';
IBQuery1=>SQL=>Add('select * from VHOD');
IBQuery1=>Open;
};
```

```
procedure TForm1.N14Click(Sender: TObject);
{
Application.Run;
};
```

```
procedure TForm1.N15Click(Sender: TObject);
{
Application=>Terminate;
};
```

```
procedure TForm1.N10Click(Sender: TObject);
{
DBGrid1=>Visible=True;
Label1=>Caption='ПОТРЕБНОСТЬ';
IBQuery1=>Close;
IBQuery1=>SQL=>Clear;
```

```
IBQuery1=>SQL=>Add('select * from  
POTREBN1');IBQuery1=>Open;  
};
```

```
procedure TForm1.N11Click(Sender: TObject);  
{  
DBGrid1=>Visible=True;  
Label1=>Caption='ПОСТАНОВКИ';  
IBQuery1=>Close;  
IBQuery1=>SQL=>Clear;  
IBQuery1=>SQL=>Add('select * from POST');  
IBQuery1=>Open;  
};
```

```
procedure TForm1.N12Click(Sender: TObject);  
{  
DBGrid1=>Visible=True;  
Label1=>Caption='3AKA3BI';  
IBQuery1=>Close;  
IBQuery1=>SQL=>Clear;  
IBQuery1=>SQL=>Add('select * from  
=>'ZAKAZ3');IBQuery1=>Open;  
};
```

```
procedure TForm1.N13Click(Sender: TObject);  
{  
DBGrid1=>Visible=True;  
Label1=>Caption='ДВИЖЕНИЕ';  
IBQuery1=>Close;  
IBQuery1=>SQL=>Clear;  
IBQuery1=>SQL=>Add('select ZAPAS.DATA, ZAPAS.SHIFR_MAT,');  
IBQuery1=>SQL=>Add('ZAPAS.OLD_ZAPAS+POSPOT3=>OUT_POSTA  
V- POSPOT3=>OUT_POTREB AS NEW_ZAPAS,');  
IBQuery1=>SQL=>Add('ZAPAS.DATA+1 AS DATA1 FROM ZAPAS,  
POSPOT3');IBQuery1=>SQL=>Add('where (ZAPAS.DATA  
=POSPOT3.OUT_DATA) AND');  
IBQuery1=>SQL=>Add('(ZAPAS.SHIFR_MAT  
=POSPOT3.OUT_SHIFR_MAT)');  
IBQuery1=>Open;  
};  
  
}.
```


Код программы генератора данных (язык Паскаль) (JIP2)

Program Generate;

```
Const
  MMAX=1
  0;
  NMAX=1
  0;
  TMAX=1
  0;
  IMAX=50
  ;
Var
  Iter: integer;
  Inter: integer;
  M: array [1..IMAX] of
  integer; N: array [1..IMAX]
  of integer; i, j, k, l: integer;
  A: array [1..TMAX, 1..MMAX, 1..NMAX] of
  double; C: array [1..NMAX] of double;
  DN: array [1..NMAX] of
  double; DV: array [1..NMAX]
  of double; DEL: array
  [1..NMAX] of double; BN:
  array [1..MMAX] of double;
  BV: array [1..MMAX] of
  double; Y: array [1..MMAX]
  of double; X: array [1..NMAX]
  of double; AX: array
  [1..MMAX] of double; DB:
  array [1..MMAX] of double;
  CX: double;
  fo: text;
  buf, buff: string;
```

```
Function Rand1(rfrom, rto: double): double;
```

```
Var d: double; i: integer;
```

```
Begin
  d:=random
  ;
  d:=rfrom+d*(rto-
  rfrom);
  i:=Round(d*10);
  d:=i;
```

```

    Rand1:=d/10
    ;
End;

Function MaxX(val: array of double; size: integer): double;
var max_val: double; i:integer;
Begin
    max_val:=val[1]
    ; for i:=1 to size
    dobegin
        if val[i]>=max_val then max_val:=val[i];
    end;
MaxX:=max_val;
End;

Function MinX(val: array of double; size: integer): double;
var min_val: double; i:integer;
Begin
    min_val:=val[1]
    ; for i:=1 to size
    dobegin
        if val[i]<=min_val then min_val:=val[i];
    end;
MinX:=min_val;
End;

Begin
Write ('Vvedite chislo vremennyh intervalov = ');
readln(Inter);Write ('Vvedite chislo reshaemyh zadach = ');
readln(Iter);
for l:=1 to Inter
dobegin
writeln('Vremennoy interval
#',l);for k:=1 to Iter do
begin
Randomize;
str(l,buff);
    str(k,buf);
    assign (fo,'out_'+buff+'_'+buf+'.txt');
    rewrite (fo);
    if l=1 then begin
        Writeln('Vvedite razmernost zadachi
#',k);If k=1 then begin
            Write ('N=');
            readln(N[k]);end else

```

```

begin N[k]:=M[k-1];
end;
Write ('M='); readln(M[k]);
writeln(fo,N[k]:2,'<==N');
writeln(fo,M[k]:2,'<==M');

if (N[k]>NMAX) or
  (M[k]>MMAX) thenbegin
  Writeln("Too big dimensions!");
  Exit
;end;
end; {if l=1 interval}

if k=1 then begin
Writeln('Vvedite
plan'); for j:=1 to N[k]
do begin
Write('X[' ,j ,']=');
  readln(X[j]);end;
end;

if (k>1) and (l>1) then
beginWriteln('Vvedite
deltaB'); for i:=1 to M[k]
do begin
Writeln('Vvedite dB dlya zadachi
#',k);Write('dB[' ,i ,']=');
  readln(DB[i]);
end;
end;

for j:=1 to N[k]
dobegin
if k=1 then begin
if X[j]<(MinX(X, N[k])+0.35*(MaxX(X, N[k])-MinX(X, N[k]))) then
begin
DN[j]:=X
[j];
DV[j]:=MaxX(X, N[k]);
DEL[j]:=Rand1(DN[j]+0.25*(DV[j]-DN[j]),DN[j]+0.45*(DV[j]-DN[j]));
end;
if (X[j]>=(MinX(X, N[k])+0.35*(MaxX(X, N[k])-MinX(X, N[k])))
AND (X[j]<=(MinX(X, N[k])+0.65*(MaxX(X, N[k])-MinX(X, N[k])))) then
begin
DN[j]:=MinX(X, N[k]);
DV[j]:=MaxX(X, N[k]);

```

```

    DEL[j]:=(MaxX(X, N[k])-MinX(X, N[k]))/2;
end;
if X[j]>(MinX(X, N[k])+0.65*(MaxX(X, N[k])-MinX(X, N[k]))) then
begin
    DN[j]:=MinX(X,
    N[k]);DV[j]:=X[j];
    DEL[j]:=Rand1(DN[j]+0.55*(DV[j]-DN[j]),DN[j]+0.75*(DV[j]-DN[j]));
end;
end else begin {if k=1 end}
    DN[j]:=BN[j];
    DV[j]:=BV[j];
    X[j]:=AX[j]+DB[j];
if X[j]<(DN[j]+0.35*(DV[j]-DN[j])) then
begin
    DEL[j]:=Rand1(DN[j]+0.25*(DV[j]-DN[j]),DN[j]+0.45*(DV[j]-DN[j]));
end;
if (X[j]>=(DN[j]+0.35*(DV[j]-DN[j]))) AND (X[j]<=(DN[j]+0.65*(DV[j]-DN[j])))
then
begin
    DEL[j]:=(DN[j]+DV[j])/
    2;

end;
if X[j]>(DN[j]+0.65*(DV[j]-DN[j])) then
begin
    DEL[j]:=Rand1(DN[j]+0.55*(DV[j]-DN[j]),DN[j]+0.75*(DV[j]-DN[j]));
end;
end; {if k!=1
end}end;

if l=1 then begin
Writeln('Vvedite matricu
A');for i:=1 to M[k] do
begin
    AX[i]:=
    0;
    for j:=1 to N[k]
    dobegin
        Write('A[' ,i ,',' ,j ,']='); readln(A[k,i,j]);
        AX[i]:=AX[i]+A[k,i,j]*X[j];
    end;
end;
end else begin
for i:=1 to M[k]
dobegin
AX[i]:=0;
for j:=1 to N[k]

```

```

    dobegin
        AX[i]:=AX[i]+A[k,i,j]*X[j];
    end;
end;
end; {if l!=1 end}

for i:=1 to M[k]
    dobegin
        if AX[i]<(MinX(AX, M[k])+0.35*(MaxX(AX, M[k])-MinX(AX, M[k]))) then
            begin
                BN[i]:=AX
                [i];
                BV[i]:=MaxX(AX, M[k]);
                Y[i]:=Rand1(BN[i]+0.25*(BV[i]-BN[i]),BN[i]+0.45*(BV[i]-BN[i]));
            end;
        if (AX[i]>=(MinX(AX, M[k])+0.35*(MaxX(AX, M[k])-MinX(AX, M[k])))
            AND (AX[i]<=(MinX(AX, M[k])+0.65*(MaxX(AX, M[k])-MinX(AX, M[k]))))
        then
            begin
                BN[i]:=MinX(AX, M[k]);
                BV[i]:=MaxX(AX, M[k]);
                Y[i]:=(MaxX(AX, M[k])-MinX(AX, M[k]))/2;
            end;
        if AX[i]>(MinX(AX, M[k])+0.65*(MaxX(AX, M[k])-MinX(AX, M[k]))) then
            begin
                BN[i]:=MinX(AX,
                M[k]);BV[i]:=AX[i];
                Y[i]:=Rand1(BN[i]+0.55*(BV[i]-BN[i]),BN[i]+0.75*(BV[i]-BN[i]));
            end;
        end;
    end;
    CX:=0;
    for j:=1 to N[k]
        dobegin
            C[j]:=DEL[j];
            for i:=1 to M[k] do
                C[j]:=C[j]+A[k,i,j]*Y[i];
                CX:=CX+C[j]*X[j];
            end;

            writeln(fo,'C, CX=',CX:6:2);
            for j:=1 to N[k] do Write(fo,C[j]:6:2,' '); Writeln(fo);
            writeln(fo,'DN');
            for j:=1 to N[k] do Write(fo,DN[j]:6:2,' ');
            Writeln(fo);writeln(fo,'DV');
            for j:=1 to N[k] do Write(fo,DV[j]:6:2,' ');
            Writeln(fo);writeln(fo,'BN');

```

```

for i:=1 to M[k] do Write(fo,BN[i]:6:2, ' '); Writeln(fo);
writeln(fo,'BV');
for i:=1 to M[k] do Write(fo,BV[i]:6:2, ' '); Writeln(fo);

writeln(fo,'A');
for i:=1 to M[k]
dobegin
    for j:=1 to N[k] do Write(fo,A[k,i,j]:6:2, ' ');
    Writeln(fo);end;
writeln(fo,'X');
for j:=1 to N[k] do Write(fo,DV[j]:6:2, ' ');
Writeln(fo);writeln(fo,'Xopt');
for j:=1 to N[k] do Write(fo,X[j]:6:2, ' ');
Writeln(fo);writeln(fo,'B');
for i:=1 to M[k] do Write(fo,AX[i]:6:2, ' ');
Writeln(fo);close(fo);
writeln;
end;
{Iteration}end;
{Interval} End.

```

Коды программ удаленного объекта (JIP3)

Листинг 1. MyServlet.java

```

import javax.servlet.GenericServlet;
import
javax.servlet.ServletException;
import
javax.servlet.ServletRequest; import
javax.servlet.ServletResponse;
import java.io.BufferedReader;
import
java.io.IOException;
import
java.io.PrintWriter;

public class MyServlet extends GenericServlet {
    public void service(ServletRequest servletRequest,
        ServletResponseservletResponse) throws ServletException,
IOException {
        servletRequest.setCharacterEncoding("UTF-8");

        BufferedReader bufferedReader =
                                servletRequest.getReader()
        ;String buffer = bufferedReader.readLine();
        bufferedReader.close();
        servletResponse.setContentType("text/html;charset=UTF-
8"); PrintWriter printWriter =
        servletResponse.getWriter();
        printWriter.println(TextPage.getPage());
        printWriter.flush();
        printWriter.close();
    }
}

```

Листинг 2. pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>

<project
                                xmlns="http://maven.apache.org/POM/4.0
.0"xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0
.0http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

```

```
<groupId>mySimpleServlet</groupId>
<artifactId>mySimpleServlet</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>war</packaging>

<name>mySimpleServlet Maven Webapp</name>
<!-- FIXME change it to the project's website -->
<url>http://www.example.com</url>

<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.source>1.7</maven.compiler.source>
  <maven.compiler.target>1.7</maven.compiler.target>
</properties>

<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.11</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>4.0.1</version>
    <scope>provided</scope>
  </dependency>

  <dependency>
    <groupId>javax.servlet.jsp</groupId>
    <artifactId>jsp-api</artifactId>
    <version>2.2</version>
    <scope>provided</scope>
  </dependency>
</dependencies>

<build>
  <finalName>mywebapp</finalName>
  <plugins>
    <plugin>
      <groupId>org.apache.tomcat.maven</groupId>
      <artifactId>tomcat7-maven-plugin</artifactId>
      <version>2.2</version>
```

```

    <configuration>
      <port>8888</port>
      <path>/</path>
    </configuration>
  </plugin>
</plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<configuration>
  <source>1.8</source>
  <target>1.8</target>
</configuration>
</plugin>
</plugins>
</build>
</project>

```

Листинг 3. web.xml

```

<!DOCTYPE web-app PUBLIC
"-//Sun Microsystems, Inc.//DTD Web Application
2.3/EN""http://java.sun.com/dtd/web-app_2_3.dtd" >

<web-app>
  <display-name>Archetype Created Web Application</display-name>

  <servlet>
    <servlet-name>MyServlet</servlet-name>
    <servlet-class>MyServlet</servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>MyServlet</servlet-name>
    <url-pattern>/servlet</url-pattern>
  </servlet-mapping>
</web-app>

```

Листинг 4. index.jsp

```

<html>
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
  <body>

    <FORM ACTION="servlet" METHOD="POST">

    Вам доступен следующий документ: <input type="submit" value="Request">
  </form>

```

```
</body>
</html>
```

Листинг 5. TextPage.java

```
public class TextPage {
    private static String textPage = "<html>\n" +
        "<body alink=\"green\">" +
        "<ol type=\"I\">\n" +
        "<li>Поступающие в аспирантуру проходят собеседование с предполагаемым научным руководителем (протокол
собеседования - двухсторонний лист).</li>\n"
+
        "<li>Личный листок по учету кадров заполняется в отделе аспирантуры, докторантуры.</li>\n" +
        "<li>Документы, необходимые для поступления в аспирантуру:</li>\n" + "</ol>\n" +
        "<ol type=\"I\">\n" +
        "    <li>заявление (двухсторонний лист);</li>\n" +
        "    <li>3 фотографии (3x4 см, матовые);</li>\n" +
        "    <li>диплом специалиста или магистра об окончании вуза и приложение к нему (оригинал или копии);</li>\n"
+
        "    <li>документ, удостоверяющий личность и гражданство поступающего;</li>\n" +
        "    <li>сведения о наличии у поступающего индивидуальных достижений
(при наличии);</li>\n" +
        "</ol>\n"
        + "<ul>\n"
        +
        "    <li>документ, подтверждающий инвалидность, для создания специальных условий при проведении вступительных
испытаний (при необходимости);</li>\n"
+
        "    <li>документ, подтверждающий прохождение практики</li>\n" +
        "    <li>документ, подтверждающий воинское звание (приписное).</li>\n" + "</ul>\n" +
        "<p>Сайт университета: <a href=\"https://bmstu.ru\">bmstu.ru</a><br>\n" +
        "Поступление                                     бакалавриат/магистратура: <a
href=\"https://bmstu.ru/admission\">bmstu.ru/admission</a>
<br>\n" +
        "Образовательный                                     портал: <a
href=\" https://e-learning.bmstu.ru\"> e-
learning.bmstu.ru </a>\n" + "</body>\n"
        + "</html>";
    public static String
        getPage(){return
            textPage;
        }
    }
```

Примеры кодов программ удаленного объекта (JIP4)

```

package slp.tcp.core.communication;

import
java.io.IOException;
import java.util.HashMap;
import java.util.Map;

import slp.tcp.core.IMessage;
import slp.tcp.core.requests.;
import slp.tcp.core.responses.;

public class MessageFactory {
    //Идентификаторы запросов
    public static final int REQUEST_LOGIN = 1;
    public static final int REQUEST_DATASET =
    2;
    public static final int REQUEST_GETALLDATASET
    = 3; public static final int
    REQUEST_GETALLWORKSHOP = 4;

    //Идентификаторы ответов
    private static final int RESPONSE_BASE = 0x40000000;
    public static final int RESPONSE_LOGIN = RESPONSE_BASE + 1;
    public static final int RESPONSE_DATASET = RESPONSE_BASE + 2;
    public static final int RESPONSE_GETALLDATASET = RESPONSE_BASE +
    3; public static final int RESPONSE_GETALLWORKSHOP =
    RESPONSE_BASE + 4;

    //Ассоциативный массив: класс сообщения => идентификатор сообщения
    private static final Map<Class<? extends IMessage>, Integer> idMap = new
    HashMap<>();

    static {
        idMap.put(LoginRequest.class, REQUEST_LOGIN);
        idMap.put(LoginResponse.class, RESPONSE_LOGIN);
        idMap.put(DatasetRequest.class, REQUEST_DATASET);
        idMap.put(DatasetResponse.class, RESPONSE_DATASET);
        idMap.put(GetAllDatasetRequest.class, REQUEST_GETALLDATASET);
        idMap.put(GetAllDatasetResponse.class, RESPONSE_GETALLDATASET);
        idMap.put(GetAllWorkshopRequest.class,
        REQUEST_GETALLWORKSHOP);
        idMap.put(GetAllWorkshopResponse.class,
        RESPONSE_GETALLWORKSHOP);
    }
}

```

```

private MessageFactory() {
}
//Создает сообщение по идентификатору
public static IMessage createMessage(int messageId) throws IOException {
    if (messageId >
        RESPONSE_BASE) {
    switch (messageId) {
        case RESPONSE_LOGIN:
            return new LoginResponse();
        case RESPONSE_DATASET:
            return new DatasetResponse();
        case RESPONSE_GETALLDATASET:
            return new GetAllDatasetResponse();
        case RESPONSE_GETALLWORKSHOP:
            return new GetAllWorkshopResponse();

        default:
            throw new IOException("Неизвестный тип сообщения: " + messageId);
    }
} else {
    switch (messageId) {
        case REQUEST_LOGIN:
            return new LoginRequest();
        case REQUEST_DATASET:
            return new DatasetRequest();
        case REQUEST_GETALLDATASET:
            return new GetAllDatasetRequest();
        case REQUEST_GETALLWORKSHOP:
            return new GetAllWorkshopRequest();

        default:
            throw new IOException("Неизвестный тип сообщения: " + messageId);
    }
}
}

public static int getMessageId(final IMessage message) {
    Integer id = idMap.get(message.getClass());
    return id.intValue();
}
}

```